

Кодирование на б... · [Подписаться на публикацию](#)

Вы читаете бесплатно по [ссылке](#) от Фариды Хана. [Станьте участником](#), чтобы получить доступ к лучшим материалам на Medium.

★ История только для участников

ИИ



Искусственный Интеллект



Наука о данных



Питон



Агент искусственного Интеллекта



Преобразование 40 миллионов документов в агентный граф знаний

929 миллионов ребер за 10,8 минуты и 83,2 % правильных ответов на 600 контрольных вопросов

59 минут чтения · 26 августа 2026 года



Фарид Хан

Подписаться



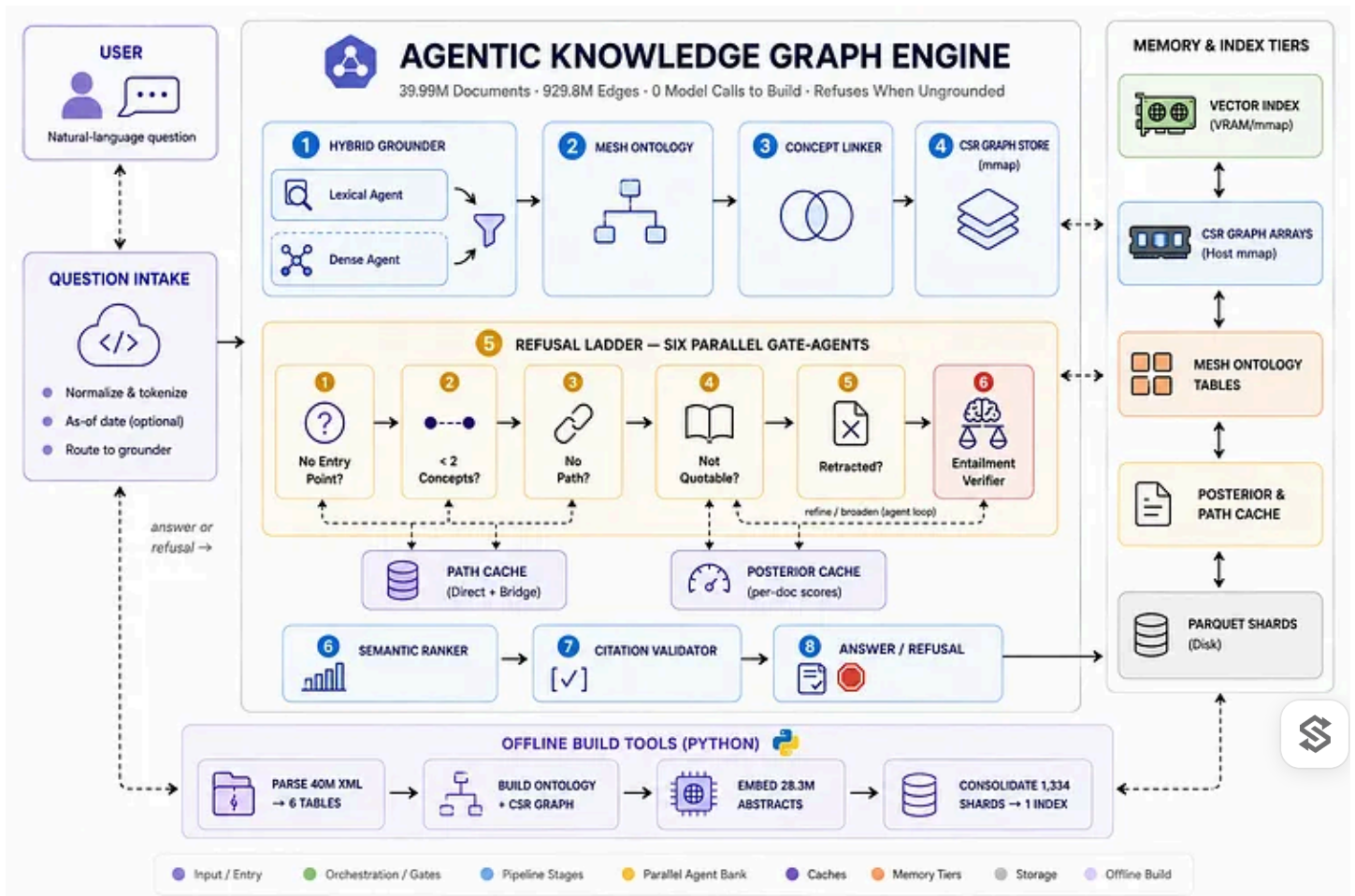
Слушай



Поделиться

Прочитайте эту статью бесплатно: [ссылка](#)

Граф знаний делает путь к ответу видимым, а агент делает этот путь точным, поскольку он проходит по нему, ссылается на него и отклоняет, если такого пути нет. Проблема возникает при работе с более чем 40 миллионами документов, поскольку обычно граф строится на основе того, что модель читает каждый документ. Поэтому индексация GraphRAG стоит около 33 000 долларов за один корпус. Но некоторые корпуса уже содержат связи, поэтому граф становится скорее результатом синтаксического анализа, чем извлечения данных, и мы получаем 929 824 202 связи за 10,8 минуты без использования моделей, используя агента, которого мы создали с нуля на Python.



Один вопрос пересекает график, а другой отклоняется еще до загрузки модели (создано [Фаридом Ханом](#))

Вот все, что мы создали: 46 модулей Python без базы данных графиков и модели извлечения данных, сверху донизу, по одному компоненту за раз:

- **Проанализировать сорок миллионов записей:** шесть столбцовых таблиц связей из 1334 сжатых XML-файлов, и ни одна строка не была выведена с помощью модели.
- **Определить размер корпуса до его создания:** сначала измеряем сохранение цитирования, потому что в недавнем срезе из десяти миллионов записей сохранилось только 28,1 % связей.
- **Создать слой онтологии:** 31 110 дескрипторов MeSH и 267 012 терминов, при этом родство задается префиксом строки, поэтому логика не требуется.
- **Сохранить граф:** сжатая разреженная строка, отображение в памяти, 2,9 миллисекунды на загрузку и 7,6 микросекунды на раскрытие узла.
- **Привязать вопрос к контексту без модели:** четыре измеренных варианта привязки и метрика, которая выявила тот вариант, который лучше всего

подходит для покрытия.

- **Пройтись по графу, а затем отказаться от него:** мосты для совместной аннотации и цитирования в один переход, а затем шесть фильтров, пять из которых — предикаты `ptru`, выполняемые до загрузки генератора.
- **Внедрить 28,3 миллиона аннотаций:** три итерации фида, от 620 до 3718 текстов в секунду, при этом менялась только подача.
- **Проверить каждое утверждение:** сначала проверяется логический вывод, потому что система, которая принимает решения об отказе, не может оценивать себя сама.
- **Создать агента, а затем отклонить его:** цикл `LangGraph` с реальной обратной связью, который каждый раз оказывался хуже обычного пайплайна.
- **Измерьте это, а потом измерьте шлейку:** 83,2 % на 600 контрольных вопросах, и одна константа в моем собственном коде, которая стоит четырнадцати баллов.



Все цифры и строки логов в этом посте получены в результате экспериментов на нашем собственном оборудовании. Весь код доступен в моем репозитории на GitHub (теория плюс код):

GitHub — FareedKhan-dev/agentic-knowledge-graph: граф знаний RAG, созданный на основе структурированных...

Граф знаний RAG, созданный на основе структурированных метаданных, а не извлеченных из больших языковых моделей...

github.com

Кодовая база организована следующим образом.

agentic-график знаний /
записная книжка /

Открыть в приложении ↗

Регистрация

Вход

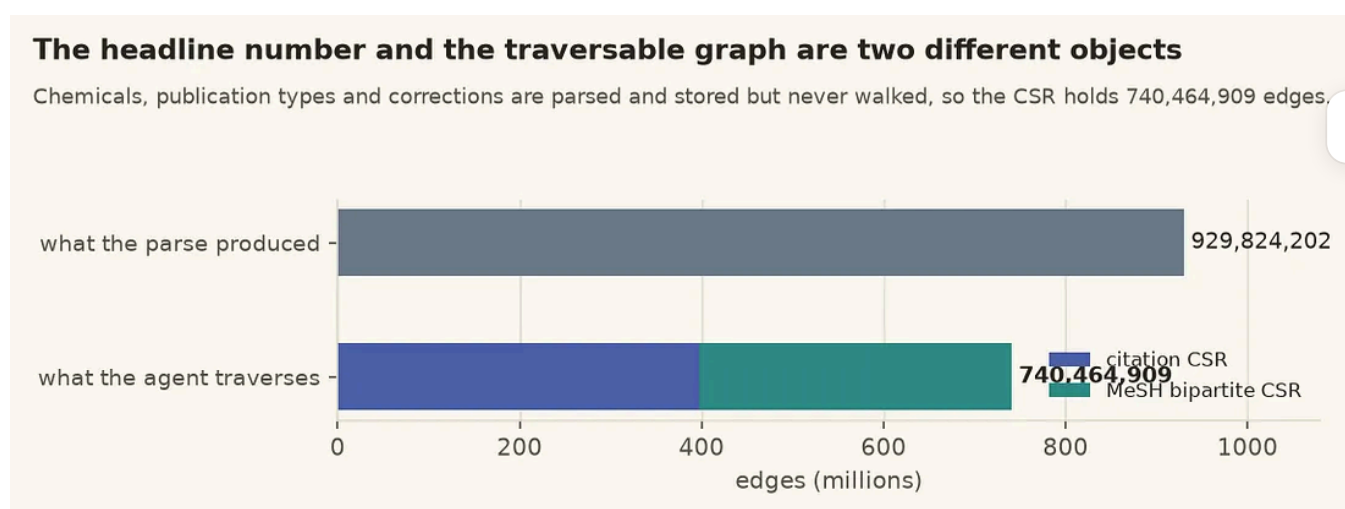
Medium

Search



```
consolidate_index.py    # 1,334 shards into one pmid-sorted memmap
kg_ground3/4/5/6.py    # the grounder, four measured versions
kg_agent.py            # the LangGraph machine, built and rejected
results/               # 32 JSON result sets and 9 posterior caches
```

Лучше заявить об этом сейчас, чем оправдываться потом. Число 929 824 202 в заголовке — это результат парсинга, а граф, по которому ходит агент, меньше. Анализируются и сохраняются химические вещества, типы публикаций и исправления, но при обходе используются только цитаты и аннотации MeSH, а это 740 464 909 ребер. Оба числа реальны, но они измеряют разные вещи.



What the parse produced, against what the agent actually walks (Created by [Fareed Khan](#))

So let us get started and build it up, one piece at a time.

The Setup, and a GPU That Reports Itself Present

Every knob lives in one parameters cell and determinism is seeded across every random source. I want the box to prove it can compute rather than merely claim it exists, so the probe multiplies two large matrices and times it.

```
import torch

assert torch.cuda.is_available(), "CUDA unavailable - check the cu128 pin"

# prove the GPU actually computes, rather than merely reporting itself present
x = torch.randn(8192, 8192, device="cuda", dtype=torch.float16)
torch.cuda.synchronize()
_t = time.time()
for _ in range(30):
```

```
y = x @ x
torch.cuda.synchronize()
print(f"  fp16 matmul      {30*2*8192**3/(time.time()-_t)/1e12:.1f} TFLOP/s")
```

```
#### OUTPUT ####
torch          2.11.0+cu128
cuda available True
device         NVIDIA H100 PCIe  sm90  79.2 GiB  114 SMs
fp16 matmul    195.1 TFLOP/s
```

195.1 TFLOP/s in half precision says the card is real and the driver stack is wired correctly. The driver is 570.195.03, which is CUDA 12.8, while current PyTorch and vLLM wheels are built against CUDA 13. Those wheels import cleanly and then report `torch.cuda.is_available() == False`, so the stack is pinned to torch 2.11.0+cu128 and vLLM is unavailable for the whole project.



The Corpus, and Why It Has To Be All Forty Million

The corpus choice is the architecture. Only PubMed gave three things at once: tens of millions of free bulk-downloadable documents, native curated edges so the graph needs no model extraction, and independent expert ground truth for every layer I measure.

One correction, because it is a thesis-level fact. **MEDLINE MeSH indexing has been machine-assigned since mid-2022**, with human curation narrowed to genes, proteins, ambiguity cases and trials. The defensible claim is curated, non-generative and versioned, a frozen classifier feeding a curated ontology, not human-curated. The genuinely NLM-curated edge families are the MeSH tree hierarchy, the citation links, and the corrections list.

The download is where I got something wrong. My first attempt used 12 parallel jobs times 4 connections, which is 48 concurrent requests, and NCBI returned HTTP 503 after 58 files. My fault, so the fix was four connections total with backoff and a missing-set recompute on every pass.

```
#### OUTPUT ####
files          1334 / 1334
total          50.5 GiB
missing        none
```

gzip-checked 10 sampled files, 0 corrupt

corpus verified

All 1,334 files landed at 50.5 GiB, none missing and none corrupt. Everything the graph is built from is visible in one record.

```
from lxml import etree

with gzip.open(files[0], "rb") as fh:
    for _, art in etree.iterparse(fh, events=("end",), tag="PubmedArticle"):
        pmid = art.findtext("./MedlineCitation/PMID")
        title = (art.findtext("./Article/ArticleTitle") or "")[:72]
        mesh = [(d.get("UI"), d.get("MajorTopicYN"), d.text)
                 for d in art.findall("./MeshHeadingList/MeshHeading/Descriptor")]
        refs = art.findall("./ReferenceList/Reference/ArticleId[@IdType='pubmed']")
        cc = [(c.get("RefType"), c.findtext("PMID"))
              for c in art.findall("./CommentsCorrectionsList/CommentsCorrection")]
        print(f" PMID          : {pmid}")
        print(f" Title           : {title}...")
        print(f" Has abstract    : {art.findtext('./Article/Abstract/AbstractText')}")
        print(f" MeSH headings   : {len(mesh)}")
        for ui, major, name in mesh[:5]:
            print(f"      {ui}      major={major} {name}")
        print(f" PubMed refs     : {len(refs)}")
        print(f" Corrections     : {cc if cc else 'none'}")
        break
```

OUTPUT

```
PMID          : 1
Title         : Formate assay in body fluids: application in methanol poisoning
Has abstract  : False
MeSH headings : 12
  D000445 major=N Aldehyde Oxidoreductases
  D000818 major=N Animals
  D001826 major=N Body Fluids
  D002245 major=N Carbon Dioxide
  D005561 major=N Formates
PubMed refs   : 0
Corrections   : none
```

Every field in that output is published by NLM, and no model produced any of it. The headings become the entity layer, the references become the citation edges, and

the corrections list becomes the supersession layer behind the retraction gate. PMID 1 has twelve headings and no abstract, which matters later when only 70.9% of the corpus is quotable.

PubMed records a reference only once NLM has resolved it to a PMID, so the edges arrive pre-resolved by the curator. That is why the retention number below comes out where it does.

Parsing Forty Million Records Into Edge Tables

This step decides whether the idea is affordable. Microsoft's published GraphRAG indexing figure is roughly **33,000 dollars in model calls for a single dataset**, three quarters of it graph extraction, which is why GraphRAG demonstrations stop at a few thousand documents.

So we extract nothing. Six tables, five of them edges, every row a field NLM already published. The parser is one function per file, and the lines that matter sit at the bottom of the loop.



```
def parse_file(path: str) -> dict:
    rows = {t: [] for t in TABLES}
    with gzip.open(path, "rb") as fh:
        ctx = etree.iterparse(fh, events=("end",), tag="PubmedArticle", huge_tree=True)
        for _, art in ctx:
            pmid = int(art.findtext("./MedlineCitation/PMID").strip())

            for mh in art.findall("./MeshHeadingList/MeshHeading"):
                d = mh.find("DescriptorName")
                major = d.get("MajorTopicYN") == "Y"
                for q in mh.findall("QualifierName") or [None]:
                    rows["mesh_edges"].append(
                        (pmid, d.get("UI"),
                         major or (q is not None and q.get("MajorTopicYN") == "Y",
                                   q.get("UI") if q is not None else None))

            for aid in art.findall("./ReferenceList//Reference//ArticleId"):
                if aid.get("IdType") == "pubmed":
                    rows["citations"].append((pmid, int(aid.text.strip())))

            # art.clear() alone is not enough, because iterparse still holds
            # every processed sibling and RSS grows with the file.
            art.clear()
            while art.getprevious() is not None:
                del art.getparent()[0]
```

The 1,334 baseline files are independent, so the work is one operating-system process per file rather than threads.

```
with Pool(args.procs) as pool:
    for r in pool.imap_unordered(parse_file, files):
        for k in tot:
            tot[k] += r[k]
```

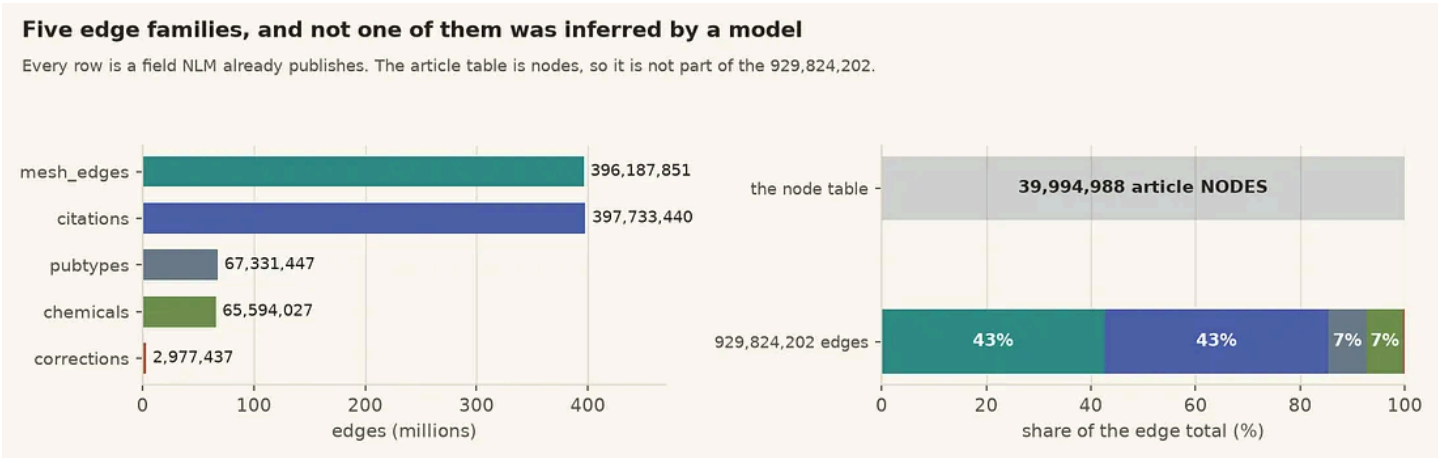
OUTPUT

table	rows	GiB	shards
articles	39,994,988	15.43	1334
mesh_edges	396,187,851	1.08	1334
citations	397,733,440	1.62	1334
corrections	2,977,437	0.02	1334
chemicals	65,594,027	0.31	1334
pubtypes	67,331,447	0.20	1334

NODES	39,994,988
EDGES	929,824,202

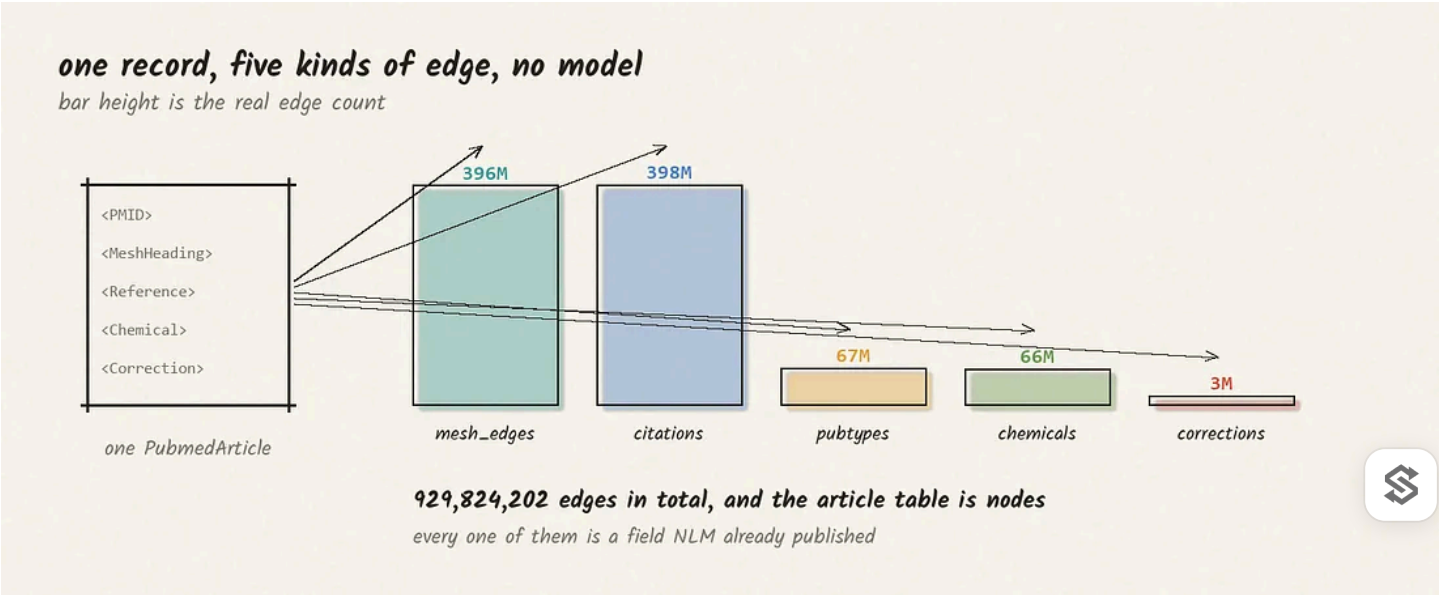
LLM calls used to build any of this: 0

Thirty nine million, nine hundred and ninety four thousand, nine hundred and eighty eight articles and 929,824,202 edges, in 10.8 minutes at 61,449 records per second, for zero model calls. The five edge families sum to the headline figure, and the article table is the node list, not a sixth family.



The five edge families, and the node table that is not one of them (Created by [Fareed Khan](#))

Against 33,000 dollars for one much smaller dataset, that gap is not an optimisation, it is a different architecture. Extraction buys relations MeSH lacks, at a price that makes forty million documents impossible.



One record, five edge families, and not one of them inferred (Created by [Fareed Khan](#))

The Decision Measurement, and Why the Corpus Is Not Ten Million

First I wanted to know whether multi-hop traversal even exists here. A citation edge is usable only if **both** endpoints sit inside the corpus we index, and low retention would mean a hairball of stubs that still scores well on every other metric.

$$\rho = \frac{|\{(u,v) \in E : u \in S \wedge v \in S\}|}{|\{(u,v) \in E : u \in S\}|}$$

Induced-subgraph retention, the number that decided the corpus size: 100 percent at 40M, 28.1 percent at 10M

Induced-subgraph retention, the number that decided the corpus size (Created by [Fareed Khan](#))

```
#### OUTPUT ####
total citation edges          397,733,440
cited_pmid inside corpus     397,731,791   (100.0%)
dangling                      1,649       (0.0%)
```

One hundred percent retention, with 1,649 dangling edges out of 397 million. PubMed records a reference only after resolving it to a PMID, so the curator already did the join.

Now the counterfactual, because the original plan was a recent ten million document slice.

```
for n_slice in (5_000_000, 10_000_000, 20_000_000):
    con.execute(f"""CREATE OR REPLACE TEMP VIEW slice AS
                SELECT pmid FROM articles ORDER BY year DESC NULLS LAST
                LIMIT {n_slice}""")
    out_e, ind = con.execute("""
        SELECT (SELECT count(*) FROM citations c
                WHERE EXISTS (SELECT 1 FROM slice s WHERE s.pmid=c.citing_pmid)
        (SELECT count(*) FROM citations c
                WHERE EXISTS (SELECT 1 FROM slice s1 WHERE s1.pmid=c.citing_pmid
                             AND EXISTS (SELECT 1 FROM slice s2 WHERE s2.pmid=c.cited_pmid)
        """).fetchone()
```

#### OUTPUT ####					
slice		outbound	induced	retained	MeSH cov

recent	5M	106,790,051	11,178,083	10.5%	58.0%
recent	10M	207,161,067	58,130,371	28.1%	63.6%
recent	20M	344,441,335	212,602,515	61.7%	72.4%
FULL	40M	397,733,440	397,731,791	100.0%	

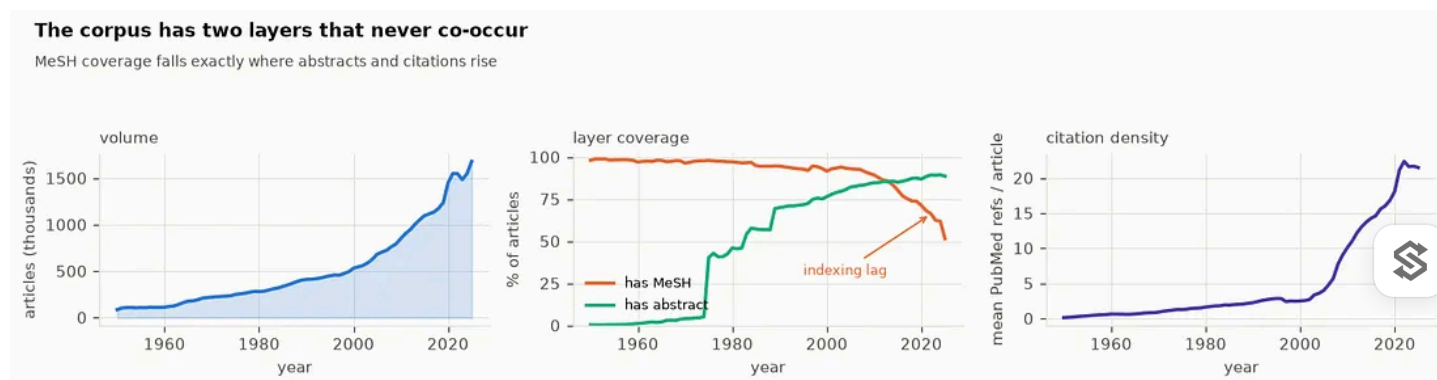
A recent ten million document slice keeps 28.1% of its citation edges and drops MeSH coverage to 63.6%. I would have built a graph of stubs and measured its refusal rate without discovering that the paths were missing.

The era breakdown shows why no slice works.

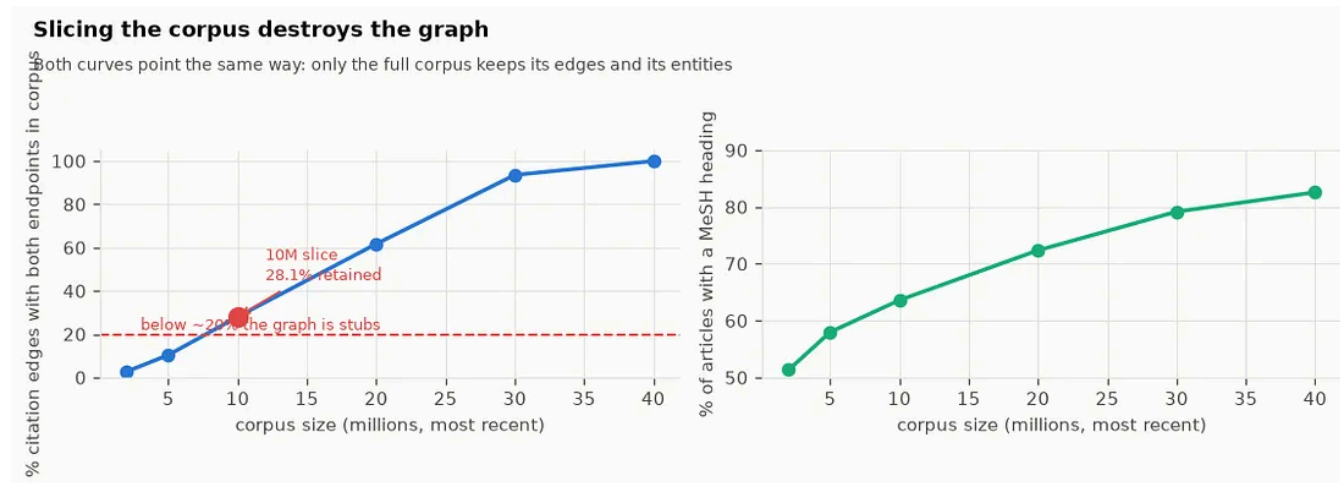
#### OUTPUT ####				
decade	articles	MeSH%	abstract%	mean refs
1950	85,773	98.0%	0.4%	0.1
1960	112,216	97.0%	1.1%	0.6
1970	219,422	96.3%	4.1%	0.9
1980	280,650	97.1%	46.0%	1.6
1990	410,654	94.5%	69.9%	2.3

2000	531,678	91.6%	76.4%	2.5
2010	849,534	89.3%	84.7%	10.1
2020	1,448,395	71.2%	86.9%	18.1

Ninety eight percent MeSH coverage in the 1950s sits beside 0.4% abstracts, and recent decades are the mirror image because indexing lags publication. The two layers never co-occur, so **only the full corpus has both at once** and going bigger was necessity.



Seventy five years of corpus, and the two layers never overlap (Created by [Fareed Khan](#))



The slice decision, drawn on the GPU box before the architecture was chosen (Created by [Fareed Khan](#))

MeSH, the Ontology We Did Not Have To Build

MeSH is not a vocabulary I picked. It is a curated hierarchical thesaurus every MEDLINE record is indexed against.

OUTPUT

```
mesh_descriptors    31,110
mesh_tree           65,360
```

mesh_broader	42,519
mesh_allowable	633,885
mesh_terms	267,012
mesh_qualifiers	76

Broader edges come from the tree, so hierarchy is free. The 633,885 allowable descriptor and qualifier pairs are a hard schema, so an edge outside that set is invalid, and the 267,012 entry terms are every synonym NLM recognises, so free text grounds without a model inventing identifiers.

Tree numbers are dotted paths, so ancestry is a string-prefix test.

$a < b \Leftrightarrow \text{tree}(a) \text{ is a dotted prefix of } \text{tree}(b)$



MeSH ancestry is a string-prefix test on dotted tree numbers, so it needs no reasoner and no transitive closure

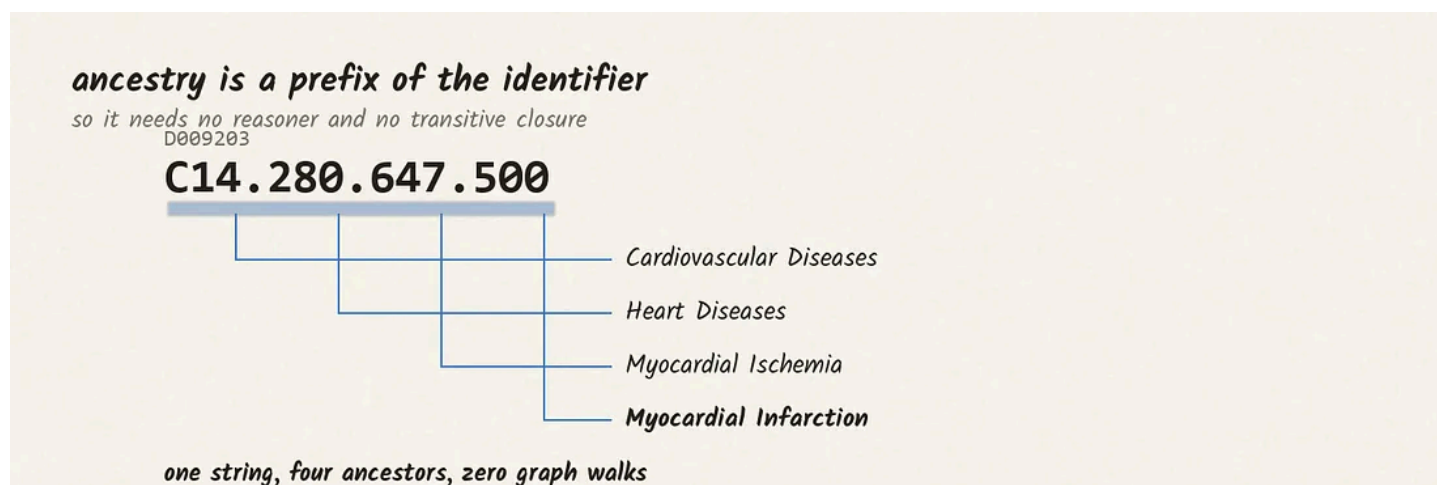
Ancestry needs no reasoner, because the identifier already encodes the path (Created by [Fareed Khan](#))

```
probe = next(u for u, n in name_of.items() if n == "Myocardial Infarction")
print(f"  '{name_of[probe]}' ({probe})  tree numbers: {ui2tn[probe]}")
print("  ancestry by STRING PREFIX - no graph walk, no Datalog:")
parts = ui2tn[probe][0].split(".")
for i in range(1, len(parts) + 1):
    anc = ".".join(parts[:i])
    if anc in tn2ui:
        print(f"      {anc:<20} {name_of.get(tn2ui[anc], '?')}")
```

OUTPUT

```
'Myocardial Infarction' (D009203)  tree numbers: ['C14.280.647.500', 'C14.907
ancestry by STRING PREFIX - no graph walk, no Datalog:
  C14                      Cardiovascular Diseases
  C14.280                  Heart Diseases
  C14.280.647              Myocardial Ischemia
  C14.280.647.500          Myocardial Infarction
```

Four ancestors recovered by slicing a string on dots. A reasoner would be slower and buy nothing, because the identifier already encodes the path.



Myocardial Infarction, four levels up, by prefix (Created by [Fareed Khan](#))



D009203 sits in four trees at once, so code assuming a single parent is wrong.

Supplementary Concepts, the Cheapest Coverage Win Available

The 31,110 main descriptors are coarse by design. MeSH has Antineoplastic Agents but not pembrolizumab, so specific drugs, rare diseases and gene products sit in a separate file.

Every supplementary record carries a `HeadingMappedTo` list pointing at main descriptors, so **it is not a new node type, it is a curated alias onto the graph that already exists**. The parser writes those aliases beside the main tables.

```
#### OUTPUT ####  
supplementary records : 324,046  
heading mappings      : 449,171  
surface terms         : 729,017
```

324,046 supplementary records contributed 729,017 raw surface terms, with no new edges and no new model. Grounding pembrolizumab now resolves through the alias, and traversal downstream is unchanged.

a supplementary record is an alias

it adds vocabulary, never a node and never an edge



729,017 surface terms in the file, 664,659 survive the guard

and the traversal machinery downstream is untouched

A supplementary record is an alias, never a new node (Created by [Fareed Khan](#))

One number gets misquoted, including by me. 729,017 is the raw file count, and only 664,659 reach the grounding index, because the loader drops aliases under eight characters and strings a main descriptor claims. One of those mapped pediatric onto a sulfonamide antibiotic.



The Graph Store, and Why It Is Not a Graph Database

I audited the graph databases before rejecting them, and the finding was simple: **there is no bulk loader anywhere.** Nodes go in one `CREATE` at a time and relationships have no batch method. At roughly eight hundred million edges and one to five milliseconds per statement, that is fourteen to fifty six hours of round trips.

Compressed sparse row solves both, because edges sorted by source make the neighbours of node i one contiguous slice.

$$\text{neighbours}(i) = \text{indices}[\text{indptr}[i] : \text{indptr}[i + 1]]$$

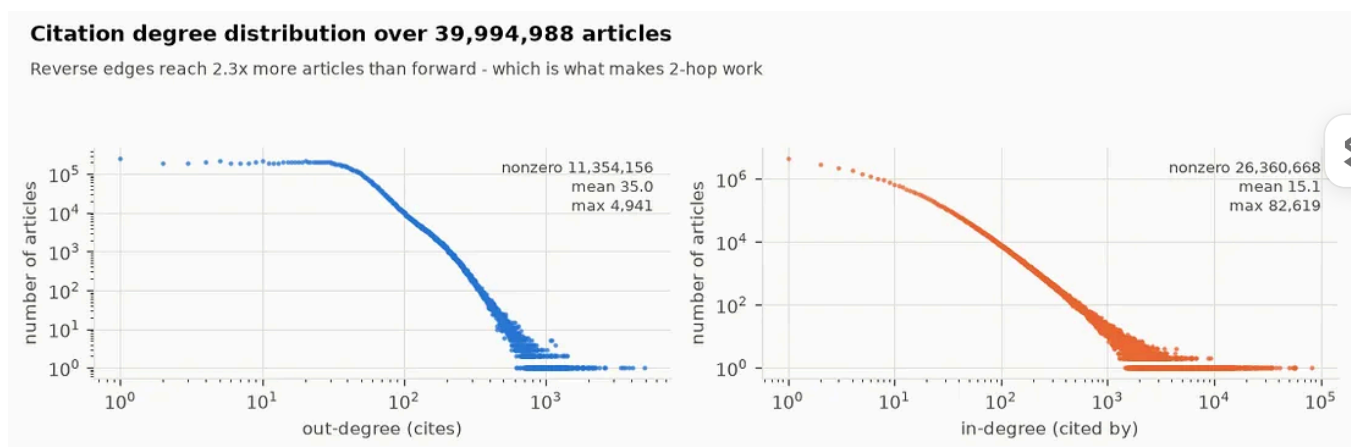
One array slice returns every neighbour of a node, so expansion costs $O(\text{degree})$ with no allocation and no query parser

The whole traversal primitive, in one array slice (Created by [Fareed Khan](#))

The build is a counting pass and a stable sort.

```
def build_csr(src, dst, n_nodes):
    counts = np.bincount(src, minlength=n_nodes).astype(np.int64)
    indptr = np.zeros(n_nodes + 1, dtype=np.int64)
    np.cumsum(counts, out=indptr[1:])
    order = np.argsort(src, kind="stable")
    return indptr, dst[order].astype(np.int32)
```

Both directions use that routine. The reverse index reaches **2.3 times more articles than the forward one**, because a paper cites a few dozen works and can be cited by thousands, and without it most of the graph is unreachable.



Citation degree over 39,994,988 articles, on log axes (Created by [Fareed Khan](#))

Every array opens memory-mapped, so construction is milliseconds and the page cache decides what stays resident.

```
class KGStore:
    def __init__(self, path=GRAPH_DIR):
        t0 = time.time()
        L = lambda n: np.load(f"{path}/{n}.npy", mmap_mode="r") # noqa: E731
        self.pmids = L("pmids")
        self.cite_indptr, self.cite_indices = L("cite_indptr"), L("cite_indices")
        self.cb_indptr, self.cb_indices = L("cited_by_indptr"), L("cited_by_indices")
        self.m2a_indptr, self.m2a_indices = L("mesh2art_indptr"), L("mesh2art_indices")
        self.N, self.D = len(self.pmids), len(self.mesh_uis)
        self.load_secs = time.time() - t0

    def cites(self, i):
        return self.cite_indices[self.cite_indptr[i]:self.cite_indptr[i + 1]]

    def cited_by(self, i):
        return self.cb_indices[self.cb_indptr[i]:self.cb_indptr[i + 1]]
```

```
def articles_of(self, d):
    return np.asarray(self.m2a_indices[self.m2a_indptr[d]:self.m2a_indptr[d
```

```
#### OUTPUT ####
load wall clock    2.9 ms
nodes              39,994,988
descriptors        31,110
citation edges     397,731,791
mesh edges        342,733,118
retracted          28,806
quotable          28,336,648  (70.9%)
```

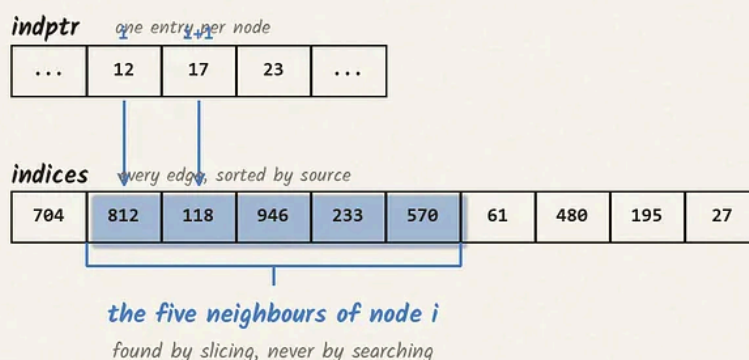
The entire graph is live in 2.9 milliseconds, because nothing is read until it is touched. The traversable graph is 397,731,791 plus 342,733,118, which is 740,464,909 edges.



Only 28,336,648 of 39,994,988 nodes are quotable, meaning they have an abstract, so the system can traverse through a node it may not cite.

one array slice IS the neighbour list

no allocation, no query parser, 7.6 microseconds a node



One index, one range, one slice of neighbours (Created by Fareed Khan)

Now the latency claim, measured.

```
probe = np.random.default_rng(0).integers(0, S.N, 20000)
_t = time.time()
touched = sum(len(S.cites(int(i))) + len(S.cited_by(int(i))) for i in probe)
_dt = time.time() - _t
```

```
print(f" 20,000 node expansions in {_dt*1000:.0f} ms")
print(f" -> {_dt/20000*1e6:.1f} us per node, {touched:,} neighbours touched")
```

OUTPUT

```
20,000 node expansions in 152 ms
-> 7.6 us per node, 396,866 neighbours touched
a Neo4j round trip is ~1-5 ms: this is ~1000x faster, in-process
```

Twenty thousand expansions in 152 milliseconds, which is 7.6 microseconds each. A database round trip is one to five milliseconds, so this is three orders of magnitude faster and in-process.

Keeping Four Hundred Million Strings Out of Numpy

My first MeSH build pulled `descriptor_ui`, a string, for all 396,187,851 rows into numpy and called `.astype(str)`. That materialises around four hundred million Python-level strings, tens of gigabytes and unbounded time.



The fix keeps the mapping in the database.

```
con.execute(f"""
    CREATE OR REPLACE TABLE dlut AS
    SELECT ui, CAST(row_number() OVER (ORDER BY ui) - 1 AS INTEGER) AS didx
    FROM read_parquet('{PQ_DIR}/mesh_descriptors/*.parquet')
""")
con.execute(f"""
    CREATE OR REPLACE TABLE me AS
    SELECT DISTINCT m.pmid AS pmid, d.didx AS didx, m.major_topic AS major
    FROM read_parquet('{PQ_DIR}/mesh_edges/*.parquet') m
    JOIN dlut d ON m.descriptor_ui = d.ui
""")
```

Joining to a 31,110-row lookup inside the database and fetching integers only turns an intractable step into a fast one. **Keep set-oriented work in the set-oriented engine, and hand numpy integers.**

The Alignment Bug That Never Raised

This is the failure mode the system exists to prevent, and I built it in myself.

NLM marks a subset of each record's headings as what the article is substantially about. Aspirin in passing and a trial of aspirin are different evidence, so that flag must live in an array aligned with the sparse row ordering.

I built it in a second pass with a different deduplication rule and no `ORDER BY`. The two arrays came out with **different row counts and different permutations**, so `major[k]` did not describe `indices[k]`.

Nothing crashed and nothing raised. Evidence would have been labelled with the wrong article's flags, in a system whose entire claim is traceability.

the flag stopped describing the edge beside it
two passes, two dedup rules, two sort orders, no error

indices	a,1	a,4	b,2	b,7	c,3	c,9	d,5
----------------	-----	-----	-----	-----	-----	-----	-----

major, pass 2	F	F	T	T	F	F	T
----------------------	---	---	---	---	---	---	---

4 of 7 wrong

major, rebuilt	T	F	F	T	T	F	F
-----------------------	---	---	---	---	---	---	---

one sorted frame plus an assert

nothing crashed, and evidence would have carried the wrong flags
in a system whose entire claim is traceability

Two passes, two orderings, and no error anywhere (Created by [Fareed Khan](#))

The rebuild derives both arrays from one sorted frame and asserts it.

```
cnt = np.bincount(dst, minlength=D).astype(np.int64)
d_indptr = np.zeros(D + 1, dtype=np.int64)
np.cumsum(cnt, out=d_indptr[1:])
o = np.lexsort((src, dst))
d_indices, d_major = src[o].astype(np.int32), maj[o]
assert len(d_indices) == len(d_major) == d_indptr[-1], "mesh2art misaligned"
```

`len(indices) = len(major) = indptr[-1]`

The invariant that was not checked the first time, so the major-topic flag did not describe the edge beside it

The invariant that was missing the first time (Created by [Fareed Khan](#))

The `lexsort` does double duty. It aligns the flags, and because the secondary key is the article index, every descriptor's article list comes out ascending, which lets intersection downstream use binary search.

An assert only proves the arrays agree with each other, so the rebuild also recounts against the database.



```
pi = int(np.searchsorted(uis, "D001241"))           # Aspirin
csr_n = int(d_indptr[pi + 1] - d_indptr[pi])
csr_maj = int(d_major[d_indptr[pi]:d_indptr[pi + 1]].sum())
db_n = con.execute("""
    SELECT count(*) FROM edge e JOIN d_lut d ON e.didx = d.didx WHERE d.ui = 'D00
""").fetchone()[0]
assert csr_n == db_n and csr_maj == db_maj, "CSR disagrees with the database"
```

```
#### OUTPUT ####
ALIGNMENT ASSERTS PASSED
Aspirin (D001241): CSR 50,506 articles (27,002 major-topic)
                   DB  50,506 articles    match=True
```

Fifty thousand five hundred and six articles on both sides, and 27,002 of them major topic. That is the check I now want on every derived structure: an internal invariant plus an independent recount from the source.

One number is worth flagging. The shipped graph reports 342,733,118 MeSH edges while the parse produced 396,187,851 raw rows, because one article and descriptor pair can repeat with different qualifiers, and the graph collapses them into one edge,

major if any row was major. The bug was never which total is correct, it was that two passes disagreed and nothing checked.

Grounding a Question Without Letting a Model Near It

No model is allowed in the grounder, because this is where a hallucination would enter. Questions map onto entry points by longest-match lookup against normalised MeSH terms, so **the model never emits an identifier and cannot invent one**.

Four versions and a new metric got it there. Version one covered 51.0% of real questions, so I categorised three hundred failures.

- 33.0% **grounded only one usable concept**, mostly British spelling and inverted preferred terms.

12.7% **had every match dropped** by an over-tight category filter that discarded whole vocabulary branches.

3.3% **matched nothing**, the classic case being circumcision against MeSH's inverted form.

Version Three: Orthography, De-inversion, and Roles

Three fixes, each aimed at a measured bucket. MeSH is US English and much medical writing is not.

```
ORTHO = [(r"oesophag", "esophag"), (r"paediatr", "pediatr"), (r"anaesthe", "anest"),
          (r"coeliac", "celiac"), (r"tumour", "tumor")] # 27 rules in all

def normalise(s):
    s = re.sub(r"^[a-z0-9 ]+", " ", unicodedata.normalize("NFKD", s.lower()))
    for a, b in ORTHO:
        s = re.sub(a, b, s)
    return " ".join(s.split())
```

The second fix is de-inversion, because MeSH holds Circumcision, Male rather than male circumcision and natural phrasing misses.

The third fix recovered most of the coverage, because **specificity is only required of a bridge**. Version two discarded every descriptor lacking it, yet Diagnosis can still narrow Keratins.

That took coverage from 51.0% to 74.0%.

Version Four, and the Metric That Caught It

Version four added supplementary aliases and head nouns, indexed only when one descriptor claims them, so circumcision is kept and diseases rejected.

Coverage went to 81.0% and I nearly shipped it, but coverage only asks whether we grounded something, which matching every word maximises.

What I want is precision against the gold article's own MeSH headings, already in my graph. **That is ground truth I did not author.**

```
def score(g, label):
    tp = fp = fn = 0
    for q, gold, gold_b in items:
        got = {S.mesh_idx(ui) for ui, _, _ in g.ground(q) if role_of(S, ui)[1]}
        tp, fp, fn = tp + len(got & gold), fp + len(got - gold), fn + len(gold - got)
    return dict(precision=tp / (tp + fp), recall=tp / (tp + fn), label=label)
```

OUTPUT

variant	precision	recall	F1	FP
v3 (orthography+deinv)	0.610	0.176	0.273	260
v4 -scr -head	0.610	0.176	0.273	260
v4 +head	0.535	0.180	0.269	361
v4 +scr	0.607	0.175	0.272	262
v4 full	0.534	0.180	0.269	363

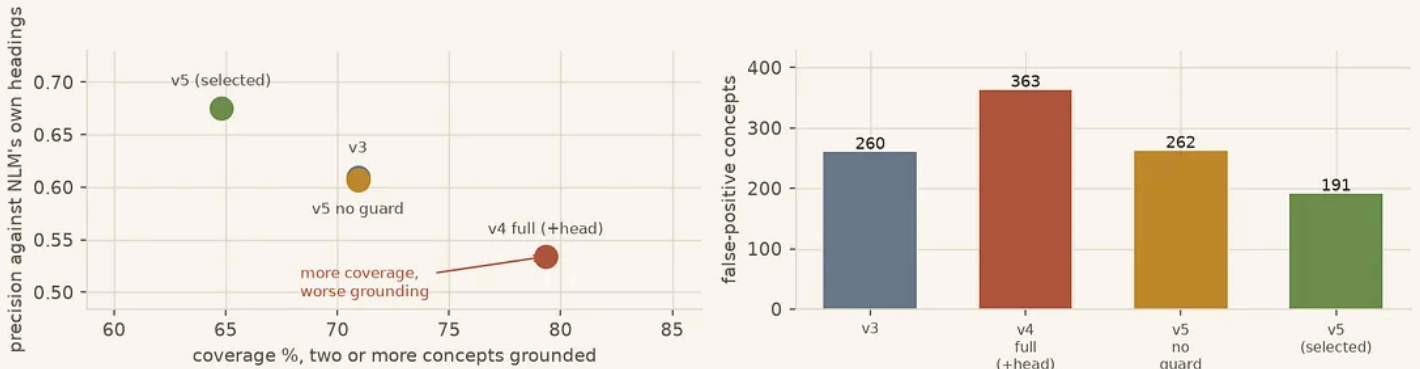
WORST SPURIOUS MATCHES (grounded, but absent from the gold article)

34x	'decrease'	-> Population Dynamics
29x	'results'	-> Reproducibility of Results
21x	'waiting'	-> Watchful Waiting
18x	'affect'	-> Affect
14x	'influence'	-> Peer Influence

Head-noun matching bought seven coverage points and cost 0.075 of precision, adding 103 false positives. The word decrease became Population Dynamics, a grammatical accident rather than a concept, so the feature was rejected.

Coverage and precision point in opposite directions

Head-noun matching bought 8 coverage points for 0.08 of precision and 103 more false positives, so it was rejected.



Coverage and precision point in opposite directions (Created by [Fareed Khan](#))

Version Five, and a Guard Built From Evidence

The final version adds one guard: a single-word match must not be ordinary English, because a multi-word phrase like myocardial infarction is already technical.

Every word in the blocked set was observed producing a false positive in the precision run. The grounder below is built once and used everywhere downstream.



```
class Grounder:
    def __init__(self, terms, names):
        self.name, self.term2ui = names, {}
        for ui, t in terms:          # descriptors, de-inverted forms, SCR aliases
            n = normalise(t)
            if len(n) >= 4:          # SCR aliases under 8: 'pediatric' -> Sulfis
                self.term2ui.setdefault(n, ui)
        self.max_words = max(len(x.split()) for x in self.term2ui)

    def ground(self, question):
        q = normalise(question).split()
        out, used = [], [False] * len(q)
        for n in range(min(self.max_words, len(q)), 0, -1):      # longest span
            for i in range(len(q) - n + 1):
                surf = " ".join(q[i:i + n])
                ui = self.term2ui.get(surf)
                if ui and not any(used[i:i + n]) and (" " in surf or surf not in
                    out.append((ui, self.name[ui], surf))
                    used[i:i + n] = [True] * n
        return out
```

```
#### OUTPUT ####
surface forms indexed : 914,356
de-inverted forms    : 97,093
```

```
supplementary aliases : 664,659
common-word guard      : 109 single words blocked
longest entry term      : 64 words
```

914,356 surface forms, of which 664,659 came from supplementary concepts and 97,093 from de-inverting MeSH's own preferred terms. The longest entry term runs sixty four words.

Let us check the orthography fix.

```
for probe in ["barrett's oesophagus", "tumour necrosis factor", "paediatric anaesthesia", "coeliac disease", "male circumcision"]:
    ui = GR.term2ui.get(normalise(probe))
    print(f" {probe:<26} -> {GR.name.get(ui, 'MISS')}")
```

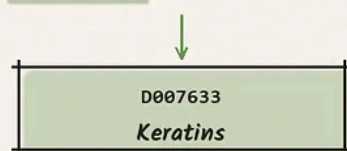
```
#### OUTPUT ####
barrett's oesophagus      -> Barrett Esophagus
tumour necrosis factor    -> Tumor Necrosis Factor-alpha
paediatric anaesthesia    -> Pediatric Anesthesia
coeliac disease           -> Celiac Disease
male circumcision         -> Circumcision, Male
```

All five resolve, and each was a miss before. The last is de-inversion, and the first four are twenty seven substitution rules earning their keep.

a question becomes graph entry points

by dictionary lookup, so no model can invent an identifier

Is cytokeratin immunoreactivity useful in the diagnosis of Barrett's oesophagus ?



cytokeratin is an entry term



oesophagus becomes esophagus

both are BRIDGE concepts, so the graph can look for what connects them

Question, spans, descriptors, and no model in the path (Created by [Fareed Khan](#))

Bridge, Filter, Ignore

A descriptor gets a role, and the rule is short.

$$\text{BRIDGE}(d) \Leftrightarrow \text{cat}(d) \in \text{ABCDEFGKJN} \wedge \text{depth}(d) \geq 3 \wedge n(d) \leq 500,000$$

A descriptor may connect two concepts only if it is deep enough and rare enough to mean something

The bridge predicate, which is the only place specificity is required (Created by [Fareed Khan](#))

```
BRIDGE, FILTER, IGNORE = "BRIDGE", "FILTER", "IGNORE"
```

```
def role_of(store, ui):
    d = store.mesh_idx(ui)
    if d < 0:
        return -1, IGNORE
    cnt, cat, dep = int(store.mesh_count[d]), str(store.mesh_cat[d]), int(store.mesh_dep[d])
    if cat == "M" or cnt > CHECK_TAG_MAX:      # Humans, Male, Adult, Animals.
        return d, IGNORE
    if cat in "ABCDEFGKJN" and dep >= 3 and cnt <= 500_000:
        return d, BRIDGE
    return d, FILTER
```

```
demo_q = "Is cytokeratin immunoreactivity useful in the diagnosis of Barrett's oesophagus?"
print(f" Q: {demo_q}")
for ui, nm, surface in GR.ground(demo_q):
    print(f" {role_of(S, ui)[1]:<7} {ui:<11} {nm:<30} <- {surface!r}")
```

```
#### OUTPUT ####
```

```
Q: Is cytokeratin immunoreactivity useful in the diagnosis of Barrett's oesophagus?
BRIDGE D001471 Barrett Esophagus <- 'barrett s esophagus'
BRIDGE D007633 Keratins <- 'cytokeratin'
```

Two bridge concepts from a question in British English using a synonym for the descriptor name. Cytokeratin is not the MeSH term, Keratins is.

The check tags are excluded for a concrete reason.

OUTPUT

most-used descriptors (all rejected as bridges):

D006801	cat=B	23,222,680 articles
D005260	cat=?	10,510,179 articles
D008297	cat=?	10,193,344 articles
D000818	cat=B	7,824,137 articles
D000328	cat=M	5,952,240 articles
D008875	cat=M	5,196,713 articles

D006801 is Humans, and it annotates 23,222,680 articles. Traversing it would connect twenty three million papers and call that evidence. The article-count rule excludes it, not the category rule, because Humans shares category B with every organism.

MeSH frequency is Zipfian: 25,836 of 31,110 descriptors are usable as bridges

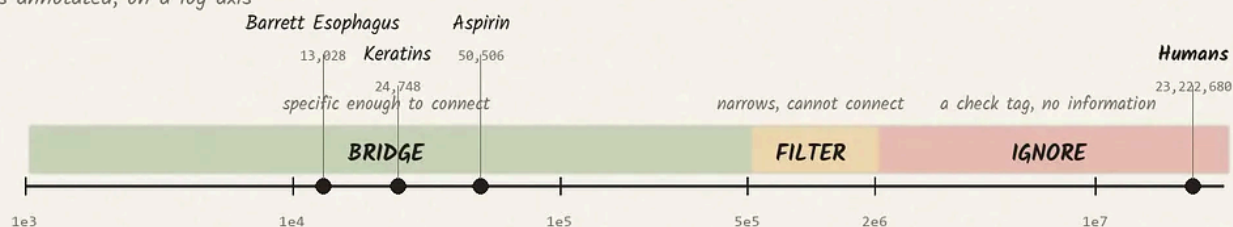
The excluded head is check tags - Humans, Male, Female, Animals



Descriptor frequency is Zipfian, and the head of it is check tags (Created by [Fareed Khan](#))

how often a descriptor is used decides what it may do

articles annotated, on a log axis



Humans annotates 23,222,680 articles, so bridging through it would connect the whole corpus

Three roles, and only one of them can connect two concepts (Created by [Fareed Khan](#))

What Actually Connects the Question

Seeding from one concept and walking outward returns aspirin papers, not aspirin-and-infarction ones. The evidence is whatever **connects** the concepts a question names, so I enumerated two classes of it, both provenanced.

```
@dataclass
class EvidencePath:
    kind: str
    hops: list
    concepts: tuple
    terminal_pmid: int
    quotable: bool
    retracted: bool
    eoc: bool
    year: int

    def render(self):
        chain = " ".join(h.render() for h in self.hops) or f"PMID {self.terminal_pmid}"
        return f"{self.kind:<6} [{ ' '.join(self.concepts)}] {chain} ({self.year})"
```



A DIRECT path is one article carrying both descriptors as major topics, zero hops and strongest because NLM asserted both. A BRIDGE path is one hop, an article on concept one citing one on concept two.

Which pair to connect when a question names three or four concepts is decided by the graph.

$$co(d_1, d_2) = |A(d_1) \cap A(d_2)|, \quad A(d) = \{a : a \text{ major topic } d\}$$

The graph picks which two concepts a question is about by counting the articles annotated with both

Pair selection is a set intersection, not a rule (Created by [Fareed Khan](#))

```
class ConceptLinker:
    def score_pairs(self, cons, verbose=False):
        """Let the GRAPH choose which concepts belong together."""
```

```

scored = []
for (d1, _, n1, _), (d2, _, n2, _) in itertools.combinations(cons, 2):
    both = np.intersect1d(self.s.articles_of(d1, major_only=True),
                          self.s.articles_of(d2, major_only=True), True)
    scored.append((len(both), d1, d2, n1, n2, both))
scored.sort(key=lambda r: -r[0])
if verbose:
    for n, _, _, n1, n2, _ in scored[:4]:
        print(f"      pair {n1!r} + {n2!r}: {n:,} co-annotated")
return scored

def link(self, groundings, verbose=False):
    cons = self.bridges(groundings)
    scored = self.score_pairs(cons, verbose)
    _, d1, d2, n1, n2, both = scored[0]
    return self._direct(both, n1, n2) + self._bridge(d1, d2, n1, n2), cons,

```

The bridge enumeration uses the ascending order the rebuild guaranteed, so membership is a binary search.



```

def _bridge(self, d1, d2, n1, n2):
    s, out = self.s, []
    A = s.articles_of(d1, major_only=True)[-self.max_seed:]
    B = s.articles_of(d2, major_only=True)
    for a in map(int, A):
        for nbrs, kind in ((s.cites(a), "CITES"), (s.cited_by(a), "CITED_BY")):
            nb = np.asarray(nbrs)
            k = np.searchsorted(B, nb).clip(0, len(B) - 1)
            for h in map(int, nb[B[k] == nb][:4]):
                out.append(self._mk("BRIDGE", [Hop(s.pmid(a), s.pmid(h), kind)]))
    return out[:self.max_paths]

```

```

q = "Does aspirin reduce the risk of myocardial infarction?"
print(f"  Q: {q}")
paths, cons, scored = CL.link(GR.ground(q), verbose=True)
n, _, _, n1, n2, _ = scored[0]
print(f"  CHOSEN: {n1!r} <-> {n2!r}  ({n:,} co-annotated articles)")
kinds = Counter(p.kind for p in paths)
print(f"  paths: {len(paths)}  (DIRECT={kinds['DIRECT']}, BRIDGE={kinds['BRIDGE']})

```

OUTPUT

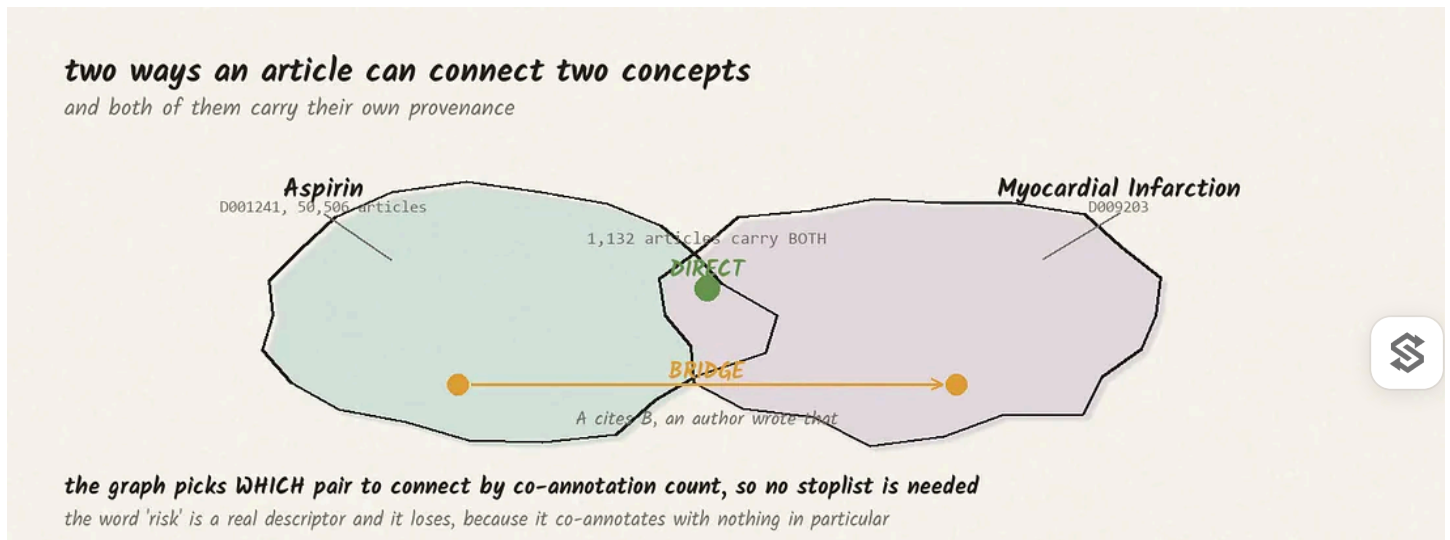
```

Q: Does aspirin reduce the risk of myocardial infarction?
  pair 'Aspirin' + 'Myocardial Infarction': 1,132 co-annotated

```

CHOSEN: 'Aspirin' <-> 'Myocardial Infarction' (1,132 co-annotated articles)
paths: 400 (DIRECT=200, BRIDGE=200)

Aspirin and Myocardial Infarction, connected by 1,132 articles carrying both as major topics. The word risk passes the specificity filter but co-annotates with almost nothing, so the graph never picks it.



Two ways to connect two concepts, and both carry their provenance (Created by [Fareed Khan](#))

Now the evidence, with its justifying path.

```
for p in sorted([p for p in paths if p.quotable and not p.retracted],
                 key=lambda p: -p.year)[:5]:
    print(f" {p.render()}")
```

OUTPUT

```
BRIDGE [Aspirin + Myocardial Infarction] 39308168 <--cited-by-- 41584723 (2020)
DIRECT [Aspirin + Myocardial Infarction] PMID 41325611 (2025)
DIRECT [Aspirin + Myocardial Infarction] PMID 41185427 (2025)
DIRECT [Aspirin + Myocardial Infarction] PMID 40888726 (2025)
DIRECT [Aspirin + Myocardial Infarction] PMID 40119267 (2025)
```

Five lines came back, and each is a defensible claim. Article 39308168 is about aspirin, article 41584723 cites it, and that one is about myocardial infarction with a quotable abstract.

The Refusal Ladder

Six gates, five of them pure graph predicates evaluated **before the generator is ever invoked**, so a refusal is an empty result set, not a model being humble.

$$\text{answer} \Leftrightarrow g_1 \wedge g_2 \wedge g_3 \wedge g_4 \wedge g_5 \wedge (|U| \geq k)$$

Every gate is a numpy predicate over the graph, so a refusal is an empty result set rather than a model declining

The gates are a conjunction, and every term is a numpy predicate (Created by [Fareed Khan](#))

```
class RefusalLadder:
    def evaluate(self, groundings, concepts, paths):
        if not groundings:
            return Verdict(False, "no_mesh_entry_point",
                           detail="nothing in the question resolves to a description")
        if len(concepts) < 2:
            return Verdict(False, "too_few_specific_concepts")
        if not paths:
            return Verdict(False, "no_path")
        u = [p for p in paths if p quotable]
        if not u:
            return Verdict(False, "no_quotable_terminal", len(paths))
        u = [p for p in u if not p.retracted and not p.eoc]
        if not u:
            return Verdict(False, "only_retracted_evidence", len(paths))
        if self.as_of:
            u = [p for p in u if 0 < p.year <= self.as_of]
            if not u:
                return Verdict(False, "no_evidence_as_of_date", len(paths), 0,
                                f"no supporting evidence existed on or before {self.as_of}")
        if len(u) < self.min_paths:
            return Verdict(False, "insufficient_paths", len(paths), len(u))
        return Verdict(True, "grounded", len(paths), len(u))
```

Gate four is not hypothetical. The graph spans forty million nodes and only 28.3 million are quotable, so the system can traverse a node it may not cite.

A helper grounds each question, evaluates the ladder, and prints the verdict. I drove five questions, two designed to fail.

OUTPUT

```
[ANSWER ] 'Does aspirin reduce the risk of myocardial infarction?'
reason=grounded paths=400 usable=323
BRIDGE [Aspirin + Myocardial Infarction] 39308168 <--cited-by-- 4158
DIRECT [Aspirin + Myocardial Infarction] PMID 41325611 (2025)

[ANSWER ] 'What is the role of metformin in type 2 diabetes mellitus?'
reason=grounded paths=400 usable=389
DIRECT [Metformin + Diabetes Mellitus, Type 2] PMID 41566384 (2026)

[ANSWER ] 'Is hydroxychloroquine effective against COVID-19?'
reason=grounded paths=362 usable=304
DIRECT [Hydroxychloroquine + COVID-19] PMID 40988009 (2025)

[REFUSE ] 'Wingardium leviosa quidditch broomstick aerodynamics'
reason=no_mesh_entry_point paths=0 usable=0
nothing in the question resolves to a descriptor
-> the generator is never called.

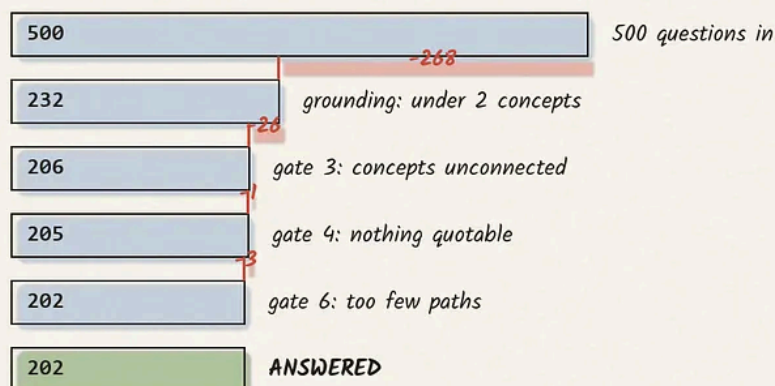
[REFUSE ] 'Is hydroxychloroquine effective against COVID-19?' [as-of 2015]
reason=no_evidence_as_of_date paths=362 usable=0
no supporting evidence existed on or before 2015
-> the generator is never called.
```



Two of the five refused. A question naming nothing in the ontology is refused at gate one, before any generation happens. Asking a 2020 question as of 2015 finds 362 paths and keeps zero, because the evidence did not exist yet.

where 500 questions actually die

band width is the number still alive



Six gates, and only the last one runs after the generator (Created by Fareed Khan)

Experiment One: Does Refusal Actually Track the Evidence?

Anything untraceable is refused is a **causal** claim, so deleting the evidence has to stop the answers. Ablation supplies a negative condition I did not label, because the deletion is the ground truth.

I made it a dose-response curve, because deleting one paper from forty million almost never removes the answer.

```
class AblatableLinker(ConceptLinker):
    def __init__(self, store, excluded: set, **kw):
        super().__init__(store, **kw)
        self.excluded = excluded

    def direct_from(self, both, n1, n2):
        both = np.array([b for b in both if int(b) not in self.excluded],
                        dtype=np.int64)
        return super().direct_from(both, n1, n2)

    def bridge(self, d1, d2, n1, n2):
        return [p for p in super().bridge(d1, d2, n1, n2)
                if self.s.idx(p.terminal_pmid) not in self.excluded]
```



The tiers are cumulative. T1 excludes the gold document, T2 adds every directly co-annotated article, T3 adds every bridge terminal.

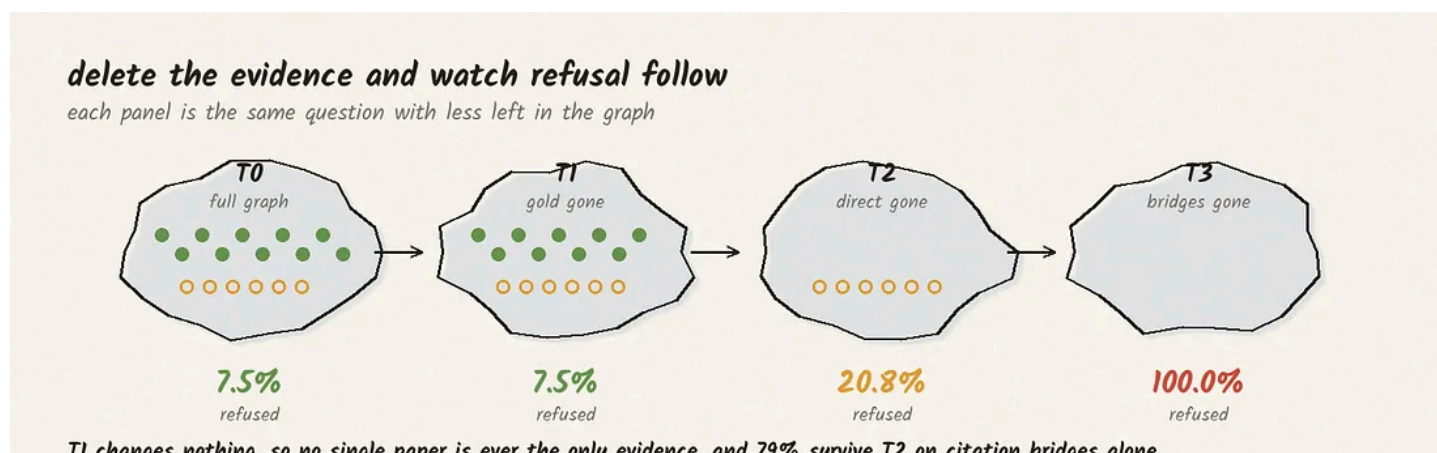
A trustworthy system is monotonic, low at T0 and high at T3. Both ends must be reported, because otherwise high refusal is indistinguishable from refusing everything.

```
#### OUTPUT ####
n = 53 PubMedQA questions (seed 42)

tier                                     refusal
-----
T0  full graph                          7.5% ###
T1  gold document removed                7.5% ###
T2  + direct co-annotated evidence removed 20.8% #####
T3  + bridge terminals removed           100.0% #####

monotonic increase : True
causal effect T3-T0: +92.5 points
```

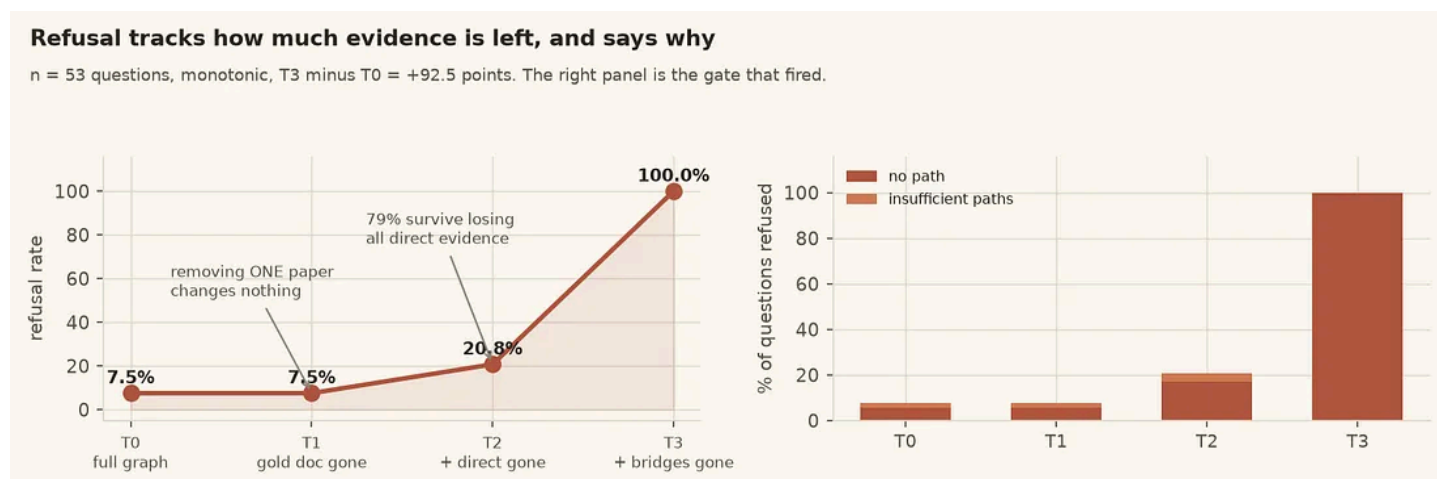
Seven and a half percent up to one hundred percent, monotonic, a causal effect of 92.5 points. Two readings inside the curve matter more.



The same question with progressively less evidence left in the graph (Created by [Fareed Khan](#))

T1 is identical to T0. Removing the single gold document changed nothing, so no paper is ever the sole evidence in forty million documents.

And **T2 is only 20.8%**, so 79% of questions survive losing all their direct evidence, because citation bridges give the graph real redundancy.



Refusal tracks how much evidence is left, and it says which gate fired (Created by [Fareed Khan](#))

T3 deletes every path and then observes refusal, which is close to definitional. The informative tiers are T1 and T2, and the defensible claim is that refusal tracks evidence availability.

Embedding 28.3 Million Abstracts, and Why the GPU Was Idle

Ranking within a path set needs semantics, so the quotable corpus gets embedded. My first loop ran at a fraction of the hardware.

A 33 million parameter model and a 110 million parameter model ran over the same texts at **the same texts per second**. The small one should be three times faster, so equal throughput means the GPU is idle.

The third version groups forty files into one super-shard, sorts it globally by length, and hands that to a persistent worker pool.

```
for grp in groups:
    all_texts, all_pmids, bounds = read_super_shard(grp)

    order = np.argsort([len(x) for x in all_texts], kind="stable")
    inv = np.empty_like(order)
    inv[order] = np.arange(len(order))

    dl = DataLoader(TextDS([all_texts[i] for i in order]),
                    batch_size=args.batch_size, num_workers=args.workers,
                    collate_fn=make_collate(tok), pin_memory=True)

    out = np.empty((len(all_texts), DIM), dtype=np.float16)
    pos = 0
    with torch.no_grad():
        for b in dl:
            b = {k: v.cuda(non_blocking=True) for k, v in b.items()}
            hid = torch.nn.functional.normalize(
                model(**b).last_hidden_state[:, 0].float(), dim=-1)
            out[pos:pos + hid.shape[0]] = hid.half().cpu().numpy()
            pos += hid.shape[0]
    out = out[inv]
```

OUTPUT

stage	txt/s	ETA 28.3M
v1 naive sentence-transformers loop	620	12.70h
v2 DataLoader workers + length-sorted	1,319	5.97h
v3 super-shards (~1M texts/loader)	3,718	2.12h
raw GPU ceiling @ seq256 bs512	13,396	0.59h

v1 naive sentence-tra	tokenisation cost 1.4x the forward pass and ran serial
v2 DataLoader workers	10 workers forked PER 13k-row shard; spawn dominated
v3 super-shards (~1M	workers amortised; length sort over a much larger pool

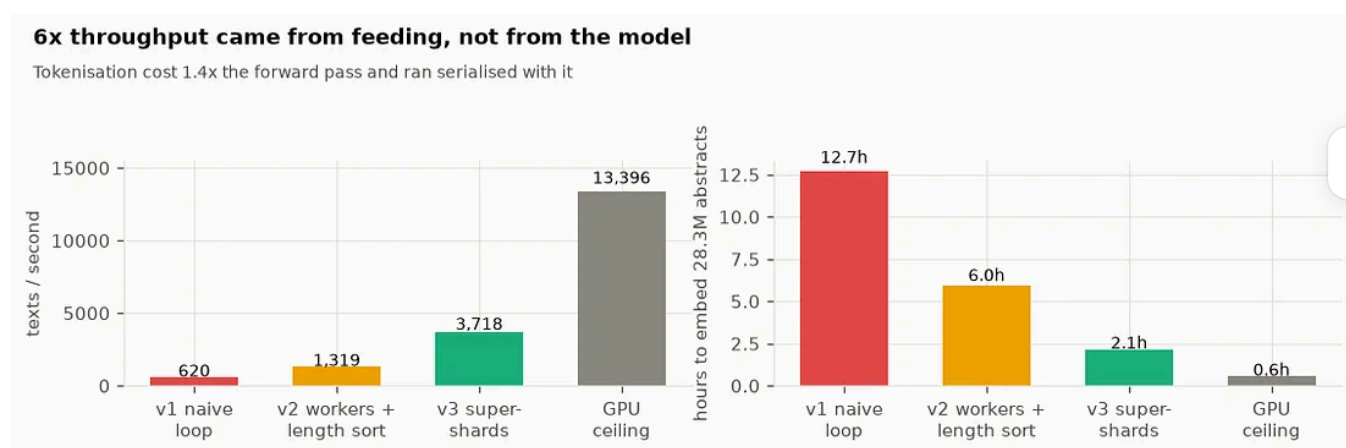
Six hundred and twenty texts per second became 3,718, six times faster, and the model never changed. Against the measured ceiling of 13,396 that is about 28%, and

the first version was under 5%. Every point came from feeding the GPU.

$$\text{ETA} = \frac{28,336,648}{r}, \quad r_{v1} = 620 \rightarrow r_{v3} = 3,718 \text{ texts/s}$$

The whole embedding job priced by the feed rate, which is what moved and not the model

The whole job priced by the feed rate (Created by [Fareed Khan](#))



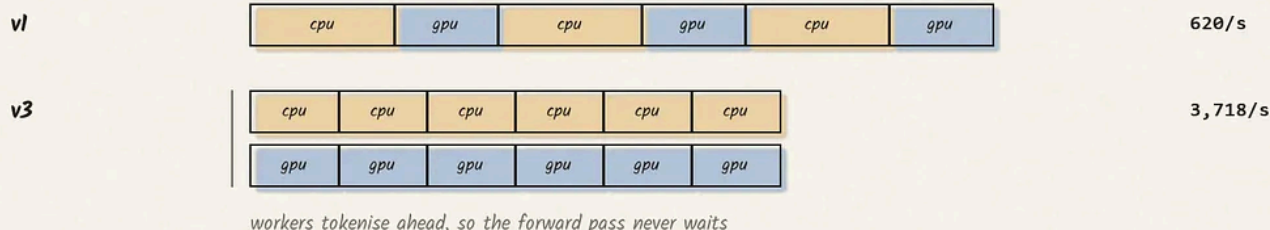
The three iterations, and the ceiling they were measured against (Created by [Fareed Khan](#))

Two bugs here were silent. **Setting** `TOKENIZERS_PARALLELISM=true` **alongside** **DataLoader workers deadlocks**, because the Rust tokenizer's thread pool does not survive a `fork()`, the symptom being fourteen live processes at zero percent GPU with no error.

The second is a query shape. `ORDER BY pmid LIMIT n OFFSET s*n` sorts 28.3 million rows once per shard. The corpus is already 1,334 Parquet files on disk, so following that layout removes the sort, matches checkpoints to the source, and lets the run survive a preemption.

the GPU was waiting, not working

one row of blocks is one second of wall clock



tokenisation cost 1.4x the forward pass and ran serialised with it

six times the throughput, and the model never changed

The GPU was never the bottleneck, the feed was (Created by [Fareed Khan](#))

Consolidating the Index, and Checking It Before Trusting It

Embedding produced 1,334 shards. The first ranker scanned all of them per question and cached the result, but keyed the cache to the **first** question's identifier set. Every later question silently scored minus one and fell back to arbitrary order.

Nothing errored. The answers simply got worse, the second time in this project a bug produced a plausible number rather than a crash, so the fix was structural.

Consolidation is two passes and never holds the index in memory. The first stable-sorts every identifier and asserts uniqueness. The second builds an inverse permutation, streams each shard into its rows of a memory-mapped output, then verifies against the sources.

```
rng = np.random.default_rng(0)
ok = 0
for _ in range(5):
    s = int(rng.integers(0, len(pmf)))
    pm = np.load(pmf[s], mmap_mode="r")
    ev = np.load(emf[s], mmap_mode="r")
    j = int(rng.integers(0, len(pm)))
    gi = int(np.searchsorted(idx, int(pm[j])))
    ok += idx[gi] == pm[j] and np.allclose(np.asarray(vec[gi]),
                                           np.asarray(ev[j]))
assert ok == 5, "consolidated index does not match its sources"
```

```
#### OUTPUT ####
shards embedded 1,334 / 1334
vectors         28,336,648 x 384d fp16
index on disk   20.3 GiB
```

28,336,648 vectors at 384 dimensions in half precision, 20.3 GiB, consolidated in 51.7 seconds. Lookup is a binary search plus one memory-mapped row read.

A separate audit compared the vector count against the quotable articles and found a gap of 2,134,898, which is 7.5% of the corpus. An index missing that much would silently degrade every ranking claim, so the audit walks four definitions of having an abstract, tests duplicate identifiers, and spot-checks sixty shards. The gap resolved, but I would not have known without looking.



one sorted index kills a whole class of bug

*the old ranker scanned every shard and cached the wrong thing
1,334 shards, each sorted only within itself*

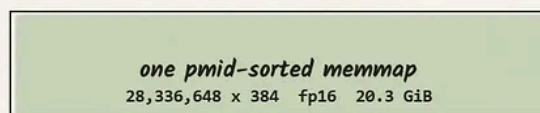


scan all of them per question

cache keyed to question 1

every later question scored -1.0

51.7 s



searchsorted

one binary search, one row read

and five random rows are checked against their source shard before anything trusts it

One thousand three hundred and thirty four shards, one sorted index (Created by [Fareed Khan](#))

Gate Six, the Propositional Verifier

Gates one to five are structural. A path proves that concepts co-occur and says **nothing** about whether the text supports the claim.

Aspirin and Myocardial Infarction co-occur in 1,132 articles whether aspirin helps, harms, or does nothing. **Polarity is not a graph property.**

$$\eta(c, d) = \text{softmax}(\text{NLI}(d, c))_{\text{neutral}}$$

Gate 6 reads the neutral mass of an entailment model, with the abstract as premise and the claim as hypothesis

Gate six is propositional. Because it decides refusals it has to be validated independently, otherwise I am measuring my verifier's opinion of my verifier.

SciFact is the right instrument, because its expert-written claims carry a no-information label, the failure mode gates one to five cannot see. I scored three models on identical pairs, with hard negatives drawn from documents a claim cites but which are not evidence.

My first evaluation had a confound. It fed gold rationale **sentences for supported and contradicted but whole abstracts for no-information**, so a model could separate the classes on length alone. At inference we never know the rationale, so every class now gets the full abstract.

Then a short adaptation run, evaluated on a dev split the model never sees.



OUTPUT

A confound we had to fix first:

v1 fed gold rationale SENTENCES **for** SUPPORT/CONTRADICT but whole ABSTRACTS **for** NOINFO - a length/specificity leak. At inference we never know the rationale, so every class now gets the full abstract.

zero-shot macro-F1 with leak : 0.597

zero-shot macro-F1 leak removed : 0.464

	zero-shot	adapted	delta
-----	-----	-----	-----
macro-F1	0.464	0.519	+0.055
NOINFO AUROC	0.722	0.932	+0.211

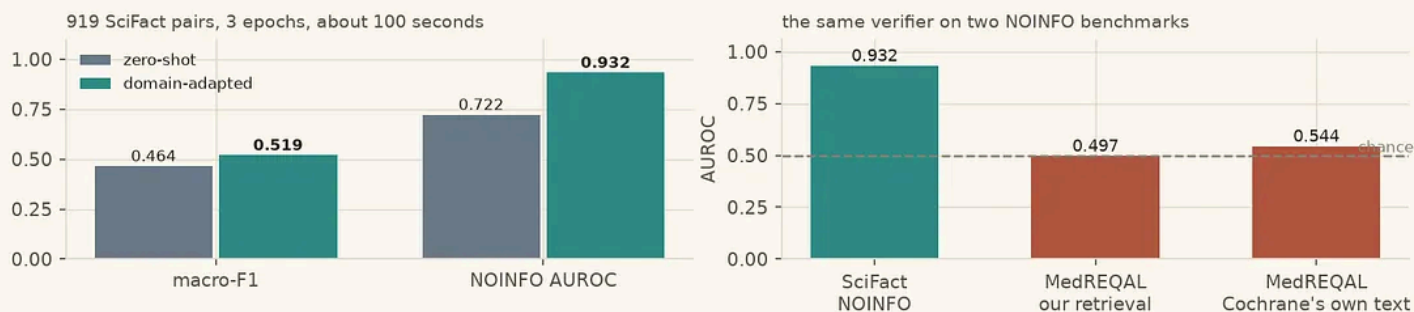
adaptation: 919 SciFact train pairs, 3 epochs, ~100 s on the H100

Removing the leak dropped zero-shot macro-F1 from 0.597 to 0.464, so a third of my first evaluation was measuring sentence length, not entailment.

Adaptation then took **no-information detection from 0.722 to 0.932 in about a hundred seconds on 919 pairs**, while macro-F1 moved only plus 0.055. The model became a far better detector of the one class gate six needs, not a better three-way classifier.

A verifier validated on one task does not transfer to another

Handing it the exact text Cochrane based its verdict on moves the AUROC from 0.497 to 0.544, which rules out retrieval as the cause.



The verifier, adapted in a hundred seconds, and where it stops working (Created by [Fareed Khan](#))

A Negative Result That Bounds the Claim

MedREQAL looked like the ideal adversarial test. Its Cochrane verdicts include a large not-enough-information class where the papers **do** exist, so gates one to five answer and gate six should catch them.



Two hypotheses, written down before the run.

- **H1:** gates one to five alone cannot separate not-enough-information from decided verdicts, so AUROC should be near 0.5.
- **H2:** gate six can separate them, so AUROC should be well above 0.5.

If H1 is false, my framing is wrong and gate six is unnecessary.

MedREQAL questions are interrogative and the verifier was trained on declarative claims, so they have to be converted, and the conversion rate gets reported rather than assumed.

OUTPUT

n = 350 questions NEI = 281

feature	AUROC vs NEI
gates 1-5: n_paths (fewer => less evidence)	0.555
gates 1-5: direct co-annotation count	0.538
gates 1-5: best-pair co-annotation	0.533
gates 1-5: refused (binary)	0.500
GATE 6: mean neutrality	0.469
GATE 6: min neutrality (best evidence)	0.497

```
GATE 6: frac abstracts neutral>0.9
```

```
0.463
```

```
H1 (structural ~ chance) : SUPPORTED (0.555)
```

```
H2 (propositional works) : REJECTED (0.497)
```

H1 held and H2 was rejected. Gate six scored 0.497 here, which is chance, despite scoring 0.932 on SciFact.

Instead of tuning I ran a diagnostic separating three explanations that demand opposite responses.

- **E1:** the verifier is broken and only looked good on SciFact.
- **E2:** the verifier is fine and we retrieved the wrong evidence.
- **E3:** the task is different in kind, so no per-document model can do it.



The discriminating test hands the verifier Cochrane's **own conclusion field**, the text the verdict was based on, rather than our retrieved abstracts. If retrieval were the problem, the score jumps.

```
#### OUTPUT ####
```

```
SciFact dev NOINFO AUROC (control re-test) : 0.932
```

```
MedREQAL via our retrieved abstracts      : 0.497
```

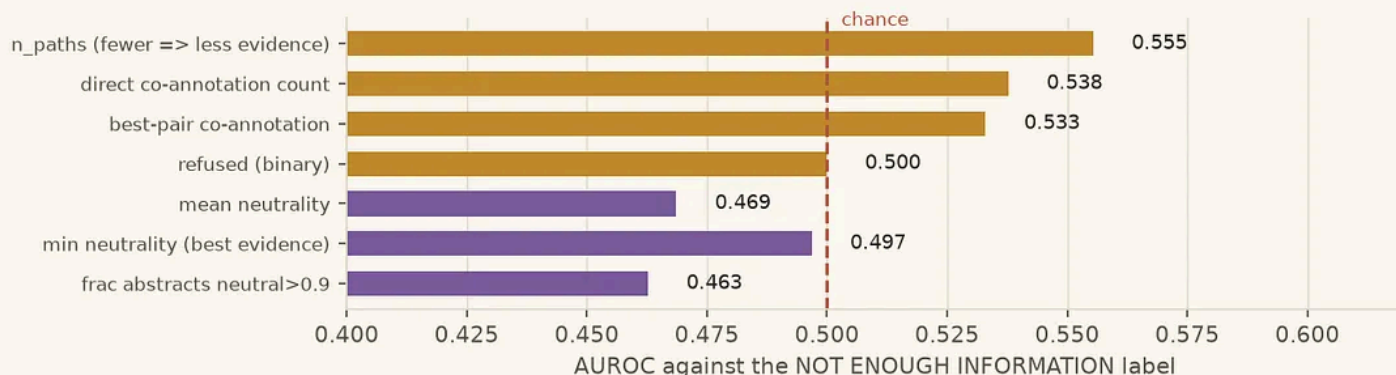
```
MedREQAL via Cochrane's OWN conclusion text : 0.544
```

The verifier is unchanged at **0.932**, which rules out E1. Given the exact text the reviewers judged from, the score moved from 0.497 to 0.544, which rules out E2.

E3 confirmed. The not-enough-information label is not an entailment judgement. It encodes risk of bias, sample size and heterogeneity across trials, a meta-analytic assessment made from full texts that no abstract-level model can recover.

Neither the structural gates nor the propositional one separates the label

n = 350 questions of which 281 are NEI. Amber is gates 1 to 5, purple is gate 6. Everything sits on chance.



Neither the structural gates nor the propositional one separates that label (Created by [Fareed Khan](#))

That bounds my thesis. Any retrieval system evaluated on MedREQAL abstention is being scored on a task it structurally cannot perform.



Closing the Loop, From Question to Cited Answer

This section produces an answer. The model never decides what is true and never decides whether to answer. The graph decides both.

five of the six gates run before the model loads

a filled dot is a stage where a model may act



the ranker re-orders what the graph produced and can never add a document, so provenance survives ranking

Who is allowed to decide what, at each of the eight stages (Created by [Fareed Khan](#))

The ranker re-orders graph output and never adds a document. An earlier version cached scores keyed to the first question's pmid set across 1,334 shards, so every later question silently scored -1.0.

$$s(q, p) = \hat{q} \cdot \hat{v}_{\text{terminal}(p)}, \quad \|\hat{q}\| = \|\hat{v}\| = 1$$

The ranker only re-orders paths the graph already produced, so it can never introduce a document

Ranking is a re-ordering, and it can never introduce a document (Created by [Fareed Khan](#))

```
class SemanticRanker:
    def rank(self, question, paths, top_k=8):
        # per call from a pmid-sorted memmap, never cached
        vecs = self.vectors_for(p.terminal_pmid for p in paths)
        qv = self.embed_query(question)
        scored = [(float(qv @ v) if (v := vecs.get(p.terminal_pmid)) is not None
                    for p in paths)]
        scored.sort(key=lambda x: -x[0])
        return scored[:top_k]
```

The prompt is constrained, and abstention is first-class.

```
ABSTAIN = "INSUFFICIENT_EVIDENCE"

SYSTEM = (
    "You answer strictly from the numbered evidence passages provided. Rules:\n"
    "1. Use ONLY facts stated in the passages. Never use outside knowledge.\n"
    f"2. If the passages do not contain the answer, reply with exactly: {ABSTAIN}\n"
    "3. Every sentence MUST end with the PMID(s) it uses, like [12345678]."
```

Citation Validation That Validated Nothing

The third silent bug made a safety claim vacuous. My parser matched one identifier per bracket, so **every identifier in a multi-citation bracket bypassed the check**, making my reported zero fabricated citations partly meaningless.

```

_CITE_BLOCK = re.compile(r"\[((?:\d{4,9})(?:\s*,\s*\d{4,9})*)\]")

def _cited_ids(text):
    return [int(x) for b in _CITE_BLOCK.findall(text)
            for x in re.split(r"\s*,\s*", b.strip())]

```

The fix keeps only identifiers in the path set.

```

def _fix(m):
    keep = [x for x in re.split(r"\s*,\s*", m.group(1).strip())
            if int(x) in valid]
    return f"[{', '.join(keep)}]" if keep else ""

clean = _CITE_BLOCK.sub(_fix, raw)

```



Demonstrated on a poisoned answer.

```

#### OUTPUT ####
valid PMIDs in path set : [40466424, 41097011]
model wrote              : Metformin works [40466424, 41097011, 999999999]. Also
after validation         : Metformin works [40466424, 41097011]. Also true [404

99999999 removed from inside a multi-citation bracket;
12345678's bracket removed entirely. Neither reaches the user.

```

The invented identifier is stripped from its bracket, and the fabricated bracket is removed.

a fabricated citation, removed from inside a bracket

the old pattern matched one id per bracket, so lists slipped past

the model wrote

Metformin works [40466424, 41097011, 999999999].

never retrieved



checked against the path set

what shipped

Metformin works [40466424, 41097011].

The Concatenation Bug That Looked Like Caution

The fourth silent bug made the system look more careful while making it worse. Gate six first scored each claim against all eight passages concatenated, but the verifier was trained on claim against one abstract, so it returned neutral almost always. The **best single source** is the in-distribution premise and the correct question.

$$\text{supported}(c) \Leftrightarrow \min_j \eta(c, d_j) < \tau, \quad \tau = 0.90$$

A claim is scored against its best single source, because the verifier was trained on one abstract at a time

The gate, scored against the best single source rather than a concatenation (Created by [Fareed Khan](#))



```
def check(self, ans, ranked, abstracts):
    srcs = [a for a in (abstracts.get(p.terminal_pmid, "")) for _, p in ranked]
    out = []
    for c in self.claims(ans.text):
        n = min((self.neutrality(c, s) for s in srcs), default=1.0)
        out.append(ClaimVerdict(c, n, n < self.tau))
    return out
```

One claim, one evidence set.

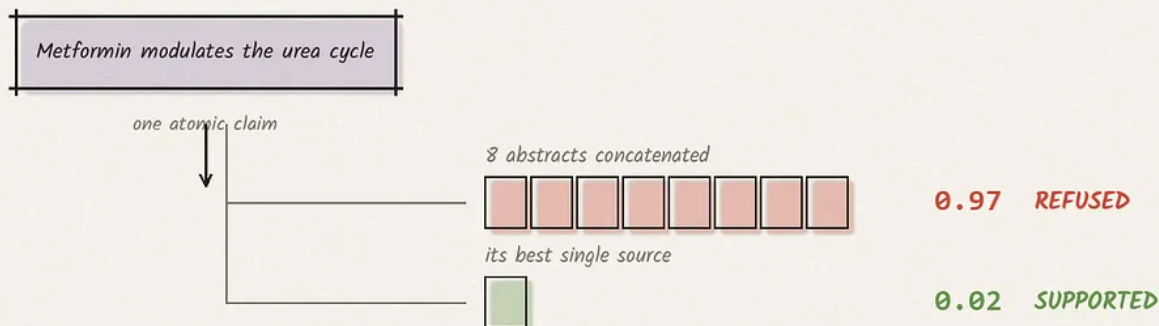
```
#### OUTPUT ####
claim: 'Metformin modulates the urea cycle.'
premise = 8 abstracts concatenated -> neutrality 0.97 -> REFUSED
premise = best single source -> neutrality 0.02 -> SUPPORTED
```

Same claim, same evidence **set**, 48x difference – purely from asking the verifier a question it was trained to answer.

A forty eight fold difference, from asking the verifier a question it was trained to answer.

the same claim, asked two different ways

the verifier was trained on one abstract at a time



a 48x difference from asking the verifier a question it was trained to answer

One claim, one evidence set, two ways of asking (Created by [Fareed Khan](#))

The Whole Loop, Running

Six questions, three of which should be refused.



OUTPUT

question	verdict	claims	secs
Does aspirin reduce the risk of myocardial inf...	ANSWERED	5	138.8
What is the role of metformin in type 2 diabet...	ANSWERED	8	78.8
Is hydroxychloroquine effective against COVID...	ANSWERED	5	24.3
Does vitamin C cure schizophrenia by quantum e...	REFUSED	0	3.1
Wingardium leviosa quidditch broomstick aerody...	REFUSED	0	0.0
Is hydroxychloroquine effective against COVID...	REFUSED	0	0.0

answered 3/6 refused 3/6

fabricated citations that reached the user: 0

Three answered, three refused, zero fabricated citations. Now one in full.

OUTPUT

Q: Does aspirin reduce the risk of myocardial infarction?

ANSWER (cited [26149039, 7438383, 1976875, 15753114]):

Yes, aspirin reduces the risk of myocardial infarction.

This is supported by the evidence from multiple studies.

For instance, the POISE-2 trial showed that perioperative aspirin did not p

Additionally, the Aspirin Myocardial Infarction Study (AMIS) found a nonsign

```
gate 6 - every atomic claim checked against its source:
OK  neutrality=0.02  Aspirin reduces the risk of myocardial infarction.
OK  neutrality=0.01  The POISE-2 trial indicated that perioperative aspirin
OK  neutrality=0.01  The Aspirin Myocardial Infarction Study (AMIS) found a
OK  neutrality=0.01  The RISC Group study demonstrated that aspirin reduced
OK  neutrality=0.04  A randomized trial of low-dose aspirin in the primary
```

Five atomic claims, each entailed by its cited abstract, with neutrality between 0.01 and 0.04 against a refusal threshold of 0.90.

Now the three refusals.

```
#### OUTPUT ####
Does vitamin C cure schizophrenia by quantum entanglement?
  refused at generation / gate 6   reason=grounded   paths=59 usable=25
Wingardium leviosa quidditch broomstick aerodynamics
  refused before generation   reason=no_mesh_entry_point   paths=0 usable=0
Is hydroxychloroquine effective against COVID-19?
  refused before generation   reason=no_evidence_as_of_date   paths=300 usable=0
```

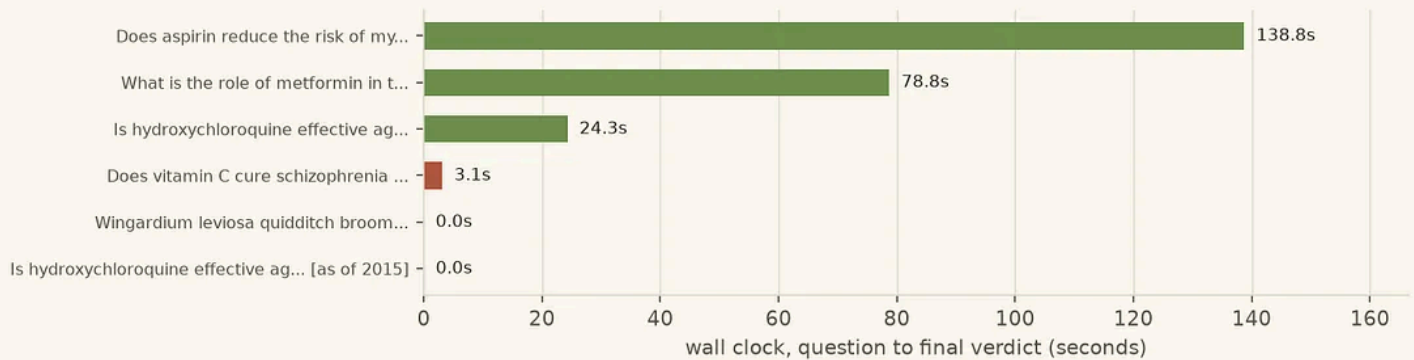
Three refusals by three completely different mechanisms. The invented spell dies at gate one, and the date-limited question finds three hundred paths and keeps zero.

The vitamin C case matters most, because the graph **correctly answers** that ascorbic acid and schizophrenia are co-studied, and the generator abstains because the claim is not supported. That is why gate six exists.

An unsupported claim is **dropped**, and the answer is refused only when nothing survives.

A refusal is cheap and an answer is not

Green answered and red refused. The two gate-1 and gate-5 refusals cost essentially nothing because the generator is never loaded into the decision.



What each question cost, from arrival to verdict (Created by [Fareed Khan](#))

The Cost Ledger

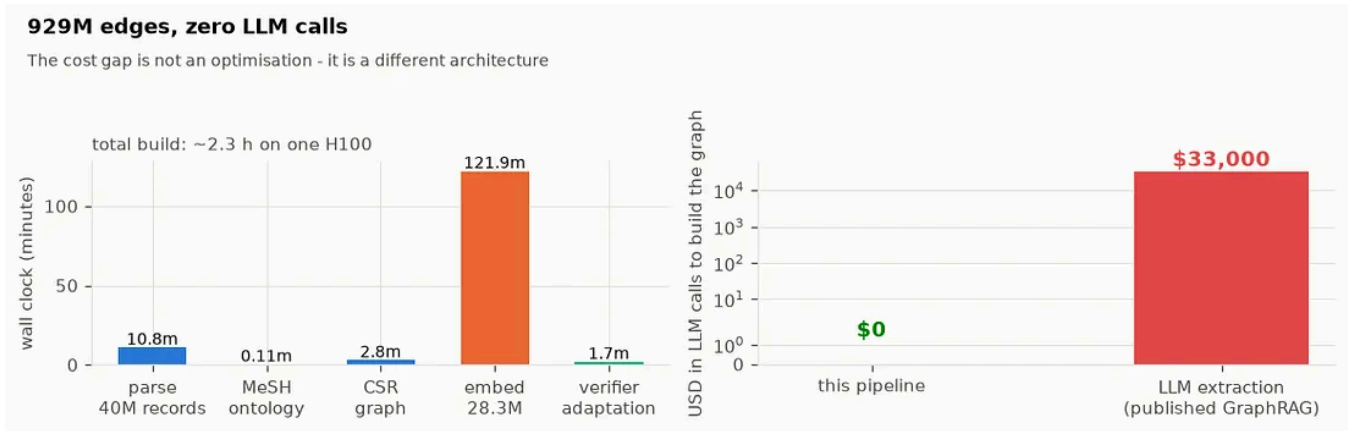
The thesis was economic before epistemic. Here is the ledger.



OUTPUT

stage	resource	time	LLM calls
acquire 50.5 GiB baseline	network	~9 min	0
parse 40M records -> 929M edges	26 CPU cores	10.8 min	0
MeSH ontology (31k desc, 634k pairs)	1 core	6.4 s	0
CSR graph build	1 core + RAM	2.8 min	0
embed 28.3M abstracts	1x H100	~2.1 h	0
verifier domain adaptation	1x H100	100 s	0
TOTAL LLM CALLS TO BUILD THE GRAPH			0

Zero model calls to build any of it, in about two and a half hours end to end including acquisition. The 2.12 hour embedding figure is a projection from the measured 3,718 texts per second, not a stopwatch reading.



What the build cost, and what the alternative costs (Created by Fareed Khan)

Where Questions Actually Die

Everything so far has been per-component. Now I want the numbers a reviewer asks for, over a large question set, starting with an **unfakeable** negative.

For each real question I built a matched control by swapping one concept for a random unrelated descriptor. It reads like a well-formed medical question, but the evidence does not exist, so the label follows from the construction, not a judgement call.



```
seed_ok_idx = np.where(np.asarray(S.seed_ok))[0]
ctrl = []
for i in pick:
    q = ds[int(i)]["question"]
    br = [(ui, nm) for ui, nm, _ in GR.ground(q) if role_of(S, ui)[1] == BRIDGE]
    if len(br) >= 2:
        ui_rand = str(S.mesh_uis[int(rng.choice(seed_ok_idx))])
        ctrl.append((int(i), q, [br[0], (ui_rand, ui_rand)]))
```

an unfakeable negative, built by permutation

so the label is a property of the construction

real question

Aspirin + Myocardial Infarction 1,132 co-annotated

↓ swap one concept for a random real descriptor

matched control

Aspirin + Snow 0 co-annotated

reads medical,
has no evidence

One concept swapped, and the label follows from the construction (Created by Fareed Khan)

A system that pattern-matches on looks medical answers both. A system that checks the graph answers only the first.

#### OUTPUT ####					
real	questions answered	40.4%	(n=500)		
control	questions answered	16.8%	(n=232)		
	discrimination	+23.6	points		
real	median co-annotated articles	0	mean	2,281	
control	median co-annotated articles	0	mean	1	

Real questions have a mean co-annotation of 2,281 articles and controls have a mean of one. The median is zero on both sides because the distribution is so skewed

The pipeline shipped with a minimum path requirement of three because three felt reasonable, which is not defensible, so I swept it.

margin = coverage – leak

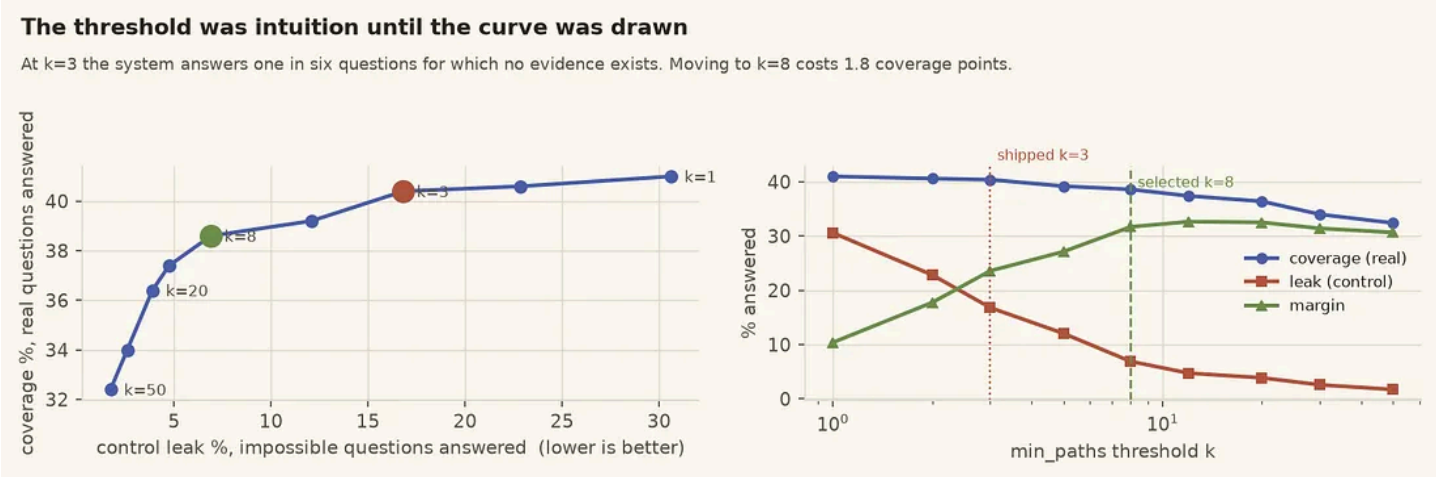
Coverage is real questions answered and leak is impossible ones answered, so only the gap is an improvement

Coverage minus leak is the only quantity that improves (Created by Fareed Khan)

#### OUTPUT ####			
k	coverage	control	leak

1	41.0%	30.6%	
2	40.6%	22.8%	
3	40.4%	16.8%	<- shipped
5	39.2%	12.1%	
8	38.6%	6.9%	<- selected
12	37.4%	4.7%	
20	36.4%	3.9%	
30	34.0%	2.6%	
50	32.4%	1.7%	

At the shipped threshold the system answers one in six questions for which no evidence exists. Moving to eight costs 1.8 coverage points and removes 59% of the leak, the difference between a system that refuses and one that mostly refuses.



The threshold was intuition until the curve was drawn (Created by [Fareed Khan](#))



The threshold moves twice more. The margin peak shifts to twelve once the grounder improves, and the final scorecard uses twenty, because leak then mattered more than coverage.

Now the funnel, which names the real limitation.

```
#### OUTPUT ####
```

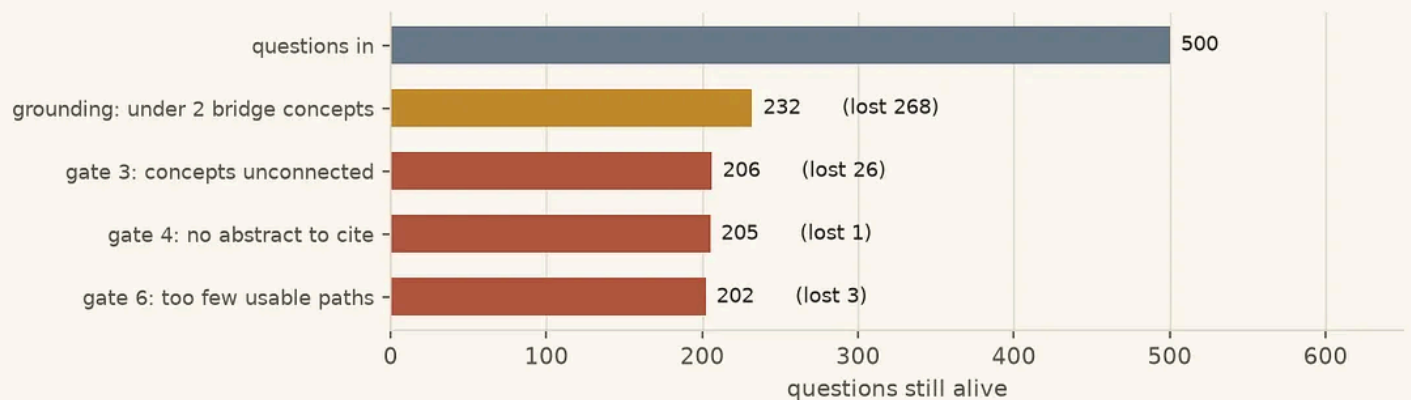
cause of loss	count	share
too_few_specific_concepts	268	53.6%
ANSWERED	202	40.4%
no_path	26	5.2%
insufficient_paths	3	0.6%
no_quotable_terminal	1	0.2%

grounding accounts for 54% of all loss;
every graph gate combined accounts for 6%.

Two hundred and sixty eight of five hundred questions never reach the graph at all, and only twenty six die because the graph has no path. Every graph gate combined accounts for six percent of the loss, so grounding is the binding constraint and I had been optimising the wrong end.

Attrition is grounding, not the gates

268 of 500 questions never reach the graph. Every graph gate combined loses 30.



Attrition is grounding, and it is not close (Created by [Fareed Khan](#))



Fixing the Binding Constraint With a Learned Linker

The lexical grounder needs an exact normalised string, and real questions paraphrase, abbreviate and use lay language. A lexical rule cannot tell circumcision from affect, so head-noun matching failed on precision.

The industry answer is a learned linker that embeds the concept vocabulary and retrieves by similarity. **The dense side has to be span-based**, because embedding a whole question conflates its concepts into one vector and a bridge needs both ends separately.

OUTPUT

concept surfaces embedded 203,462 x 384
descriptors covered 31,110
index on disk 149 MiB

What the dense side recovers that the lexical side cannot:

'should circumcision be performed in childhood'	-> Circumcision, Male	(
'sugar diabetes pills'	-> Hypoglycemic Agents	(
'water pill for high blood pressure'	-> Antihypertensive Agents	(
'PICU length of stay prediction'	-> Length of Stay	(
'keyhole surgery complications'	-> Intraoperative Compl.	(

Water pill resolves to Antihypertensive Agents and keyhole surgery to Intraoperative Complications. No lexical rule reaches those, and 149 MiB is a small

price.

Now the hybrid grounder. Function words never start or end a span, because fragments like of the heart embed to plausible nonsense.

```
class HybridGrounder:
    """Lexical is certain. Dense is admitted only above a calibrated threshold,
    and never overwrites a lexical hit on the same span."""

    EDGE_STOP = {"a", "an", "the", "of", "in", "on", "for", "to", "with", "by",
                  "is", "are", "and", "or", "that", "as", "from", "can", "not"}

    def _spans(self, words):
        out = []
        for n in range(1, min(self.max_span, len(words)) + 1):
            for i in range(len(words) - n + 1):
                sp = words[i:i + n]
                if sp[0] in EDGE_STOP or sp[-1] in EDGE_STOP:
                    continue
                if len(" ".join(sp)) >= 4:
                    out.append((i, i + n, " ".join(sp)))
        return out
```



Scoring every span against every surface in one batch is where the GPU earns its place, one scatter_reduce collapsing 203,462 surfaces onto 31,110 descriptors.

```
@torch.no_grad()
def _dense(self, spans):
    e = self.tok([s for _, _, s in spans], return_tensors="pt", padding=True,
                  truncation=True, max_length=32).to("cuda")
    qv = torch.nn.functional.normalize(
        self.enc(**e).last_hidden_state[:, 0].float(), dim=-1).half()
    sims = qv @ self.surf.T
    best = torch.full((len(spans), self.n_desc), -1.0, device="cuda",
                      dtype=torch.float16)
    best.scatter_reduce_(1, self.owner.expand(len(spans), -1), sims, reduce='max')
    top = torch.topk(best, self.top_per_span, dim=1)
    return top.values.float().cpu().numpy(), top.indices.cpu().numpy()
```

And now the result that nearly killed the whole thing.

OUTPUT

variant	precision	recall	coverage
v5 lexical	0.675	0.172	64.8%
hybrid tau=0.9	0.585	0.204	83.2%
hybrid tau=0.86	0.458	0.217	93.9%
hybrid tau=0.82	0.412	0.216	95.9%
hybrid tau=0.78	0.400	0.214	97.4%

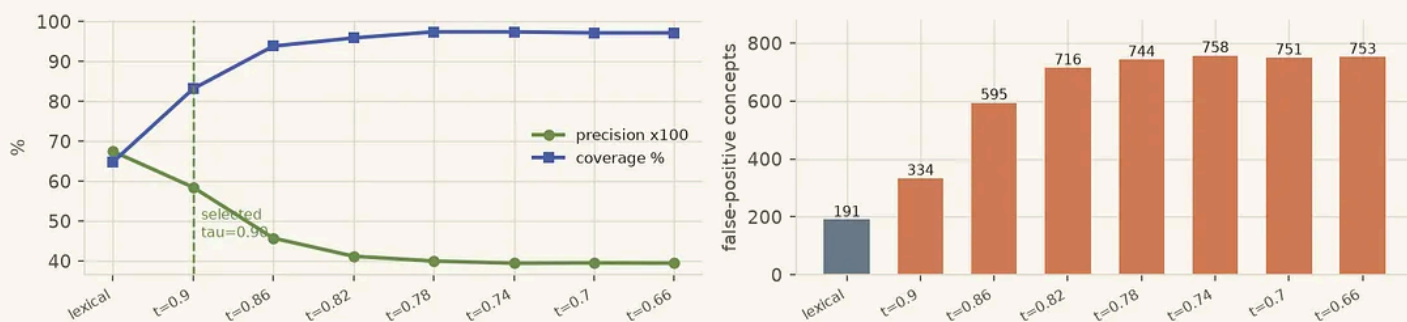
No threshold preserves precision. Component-wise, the dense linker looks like a mistake - it grounds 'winter' to Snow at 0.90.

Precision falls from 0.675 to 0.585 at the tightest threshold and keeps falling. On the component metric that is disqualifying, and I had just argued that component metrics catch mistakes.



Lowering the dense threshold buys coverage with precision

The collapse is the very first step, 0.90 down to 0.86, which nearly doubles the false positives for 10 more coverage points.



Lowering the threshold buys coverage with precision (Created by [Fareed Khan](#))

Why the Component Metric Was the Wrong Question

A spurious concept only harms if it survives the co-annotation pair selection downstream, and winter grounding to Snow co-annotates with nothing in the question, so the graph discards it unprompted.

The correct test is therefore system-level, with both halves reported together. Does coverage rise, and does leak rise with it?

OUTPUT

grounder	coverage	leak	margin
lexical (v5)	37.0%	10.5%	26.5

hybrid tau=0.90	51.0%	9.1%	41.9
hybrid tau=0.86	59.8%	10.9%	48.8

hybrid tau=0.90: coverage +14.0, leak -1.4

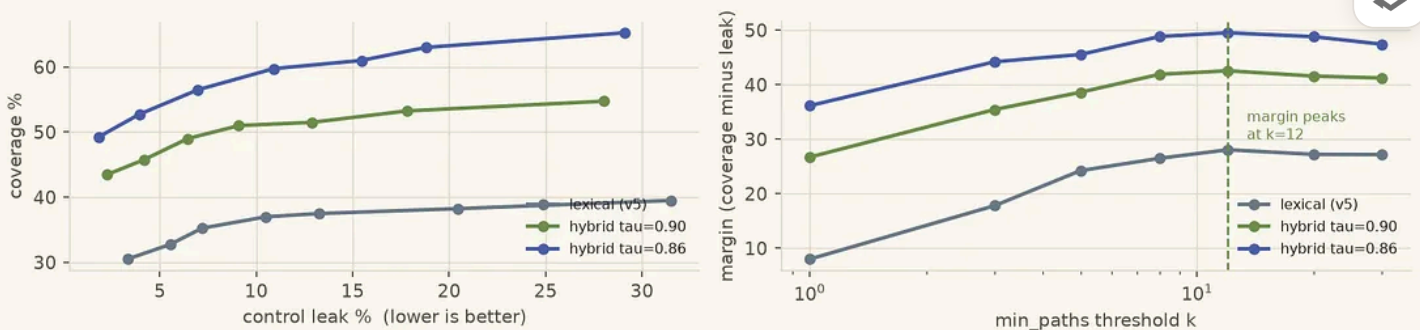
hybrid tau=0.86: coverage +22.8, leak +0.4

tau=0.90 raises coverage +14.0 AND LOWERS leak -1.4: strictly better on both axes. Component precision was measuring the wrong thing.

Coverage rises fourteen points and leak falls 1.4 points, which is strictly better on both axes at once. A grounder that raised both equally would have bought nothing, and the component metric pointed the wrong way.

The dense linker loses on precision and wins on margin

Component precision does not model the co-annotation filter downstream, which discards a spurious concept without being told to.



The linker loses on precision and wins on margin (Created by [Fareed Khan](#))

The lesson is not that component metrics do not matter. It is that **a component metric is only valid if it models what happens downstream**, and mine did not model a filter sitting in the pipeline.

The Agent, Built and Measured and Rejected

Up to here the system is a pipeline with a guard, and calling it an agent would overclaim. A genuine agent adds a **feedback edge** that acts on the graph's own structure when evidence is insufficient, so I built one as a state machine with seven nodes and one loop.

```
def _build(self):
    g = StateGraph(AgentState)
    for n, f in NODES:
        g.add_node(n, f)
    g.add_node("dead", dead_node)

    # plan, retrieve, grade, refine,
    # generate, verify, adjudicate,
    # finalize
```

```

g.set_entry_point("plan")
g.add_conditional_edges("plan", self._after_plan,
                        {"retrieve": "retrieve", "dead": "dead"})
g.add_conditional_edges("grade", self._after_grade,
                        {"generate": "generate", "refine": "refine",
                         "dead": "dead"})
for a, b in [("retrieve", "grade"),
             ("refine", "retrieve"),           # <- the feedback edge
             ("generate", "verify"), ("verify", "adjudicate"),
             ("adjudicate", "finalize"), ("dead", "finalize"),
             ("finalize", END)]:
    g.add_edge(a, b)
return g.compile()

```

The refinement node has three strategies, in order of principle.



```

def n_refine(self, st: AgentState) -> AgentState:
    st["hops"] += 1
    applied = None
    if "broaden" not in st["strategy"]: # 1: broaden up the MeSH tree
        cons = sorted(st["concepts"], # most specific concept first
                      key=lambda c: int(self.S.mesh_count[self.S.mesh_idx(c)
        for ui, nm in cons:
            anc = [a for a in self.tree.parents(ui) if self.S.mesh_idx(a) >= self.S.mesh_idx(ui)]
            if not anc:
                continue
            pname = self.GR.name.get(anc[0], anc[0])
            st["concepts"] = [(anc[0], pname) if u == ui else (u, n)
                              for u, n in st["concepts"]]
            applied = f"broaden {nm} -> {pname}"
            st["strategy"].append("broaden")
            break
    # 2: drop the weakest concept. 3: halve min_paths ONCE.
    st["trace"].append(("refine", applied or "no strategy left"))
    return st

```

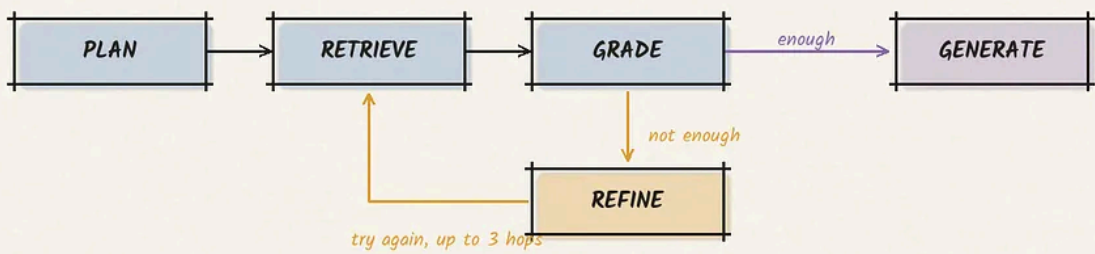
Broadening is the principled one, walking up curated **BROADER** edges so a rare drug becomes its drug class, and every broadened concept keeps real provenance.

Relaxing is unprincipled, so it happens once per question and the trace records it.

The loop has to recover real questions without answering more impossible ones. A loop that keeps trying until something comes back is a slower way to be wrong.

the loop that did not pay for itself

of the questions it answered, 69 of 73 used no refinement at all



broaden, repair, relax. None of them fixes a question that grounded fewer than two concepts.

The loop, the three strategies, and the one edge that makes it an agent (Created by [Fareed Khan](#))

OUTPUT

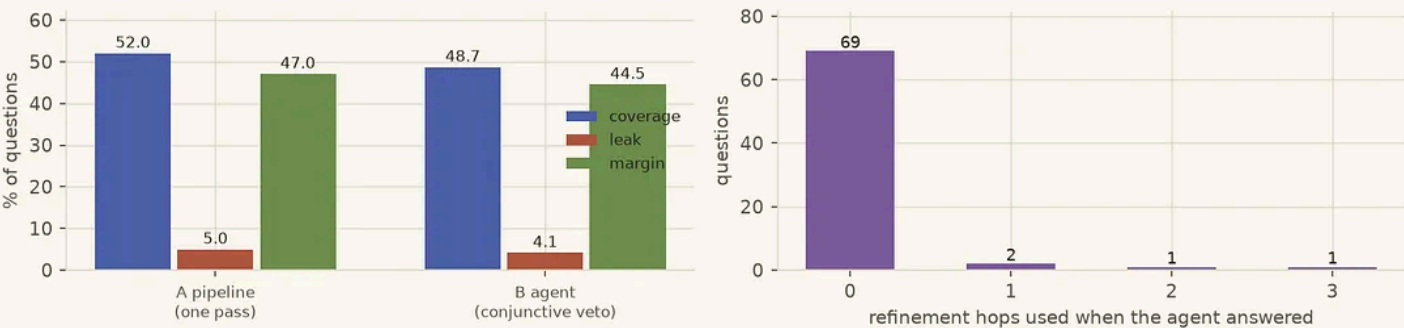
arm	coverage	leak	margin
A pipeline (one pass)	52.0%	5.0%	47.0
B agent (conjunctive veto)	48.7%	4.1%	44.5

rescued real questions : 2 of 72 refused
newly leaked controls : 3
hops used when answering: {0: 69, 1: 2, 2: 1, 3: 1}

The agent rescued two real questions and newly leaked three controls, and its margin is 2.5 points worse than the pipeline it was supposed to improve. Of the questions it answered, sixty nine used no refinement at all.

The refine loop cost more margin than it bought

It rescued 2 real questions and newly leaked 3 controls, and 69 of its answers used no refinement at all.



The loop cost more margin than it bought (Created by [Fareed Khan](#))

When the pipeline refuses, it is overwhelmingly because grounding yielded fewer than two concepts, or because the two concepts are genuinely not co-studied.

Broadening fixes neither. One concept broadened is still one concept, and unrelated parents that are co-studied give evidence about the parents. The loop treats a downstream symptom of an upstream failure.

The second error was mine. My first two versions folded semantic relevance into a **weighted average at 0.35**, which lets a strong structural score compensate for irrelevant evidence, exactly what a broadened control pair produces.

```
def _after_grade(self, st):
    """Conjunctive gating. Semantic relevance is a CONSTRAINT, not a term."""
    struct_ok = st["grade"] >= self.grade_ok and st["n_usable"] >= self.mi
    if struct_ok and st.get("semantic", 0.0) >= self.tau_sem:
        return "generate"
    return "refine" if st["hops"] < self.max_hops else "dead"
```



I had built a score where a constraint was needed, because a gate has to be able to veto. Fixing that cut the leak from 5.0 to 4.1 and cost more coverage than it saved, which is how the agent row ends up worse.

The Signal That Does Work

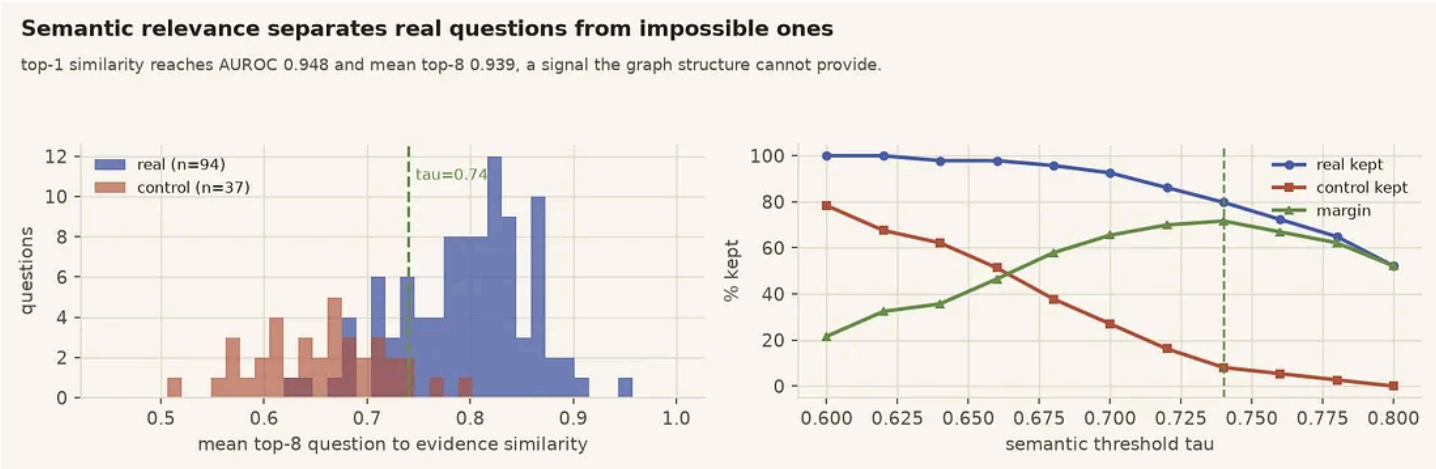
Broadening raises co-annotation for real and random pairs alike, destroying discrimination. Question-to-evidence similarity is **independent of graph structure**, because a control pair retrieves abstracts that are not about the question however broadly we search.

OUTPUT

signal	real	control	AUROC
co-annotation (graph)	2841.032	3.892	0.942
top-1 similarity	0.824	0.669	0.948
mean top-8 similarity	0.795	0.655	0.939

Top-1 similarity separates real questions from impossible ones at AUROC 0.948, a signal the graph structurally cannot provide. It went into the agent as the conjunctive

veto and still did not rescue the loop, because it was fixing the wrong stage.



A signal the graph cannot provide, on a population the graph cannot separate (Created by [Fareed Khan](#))

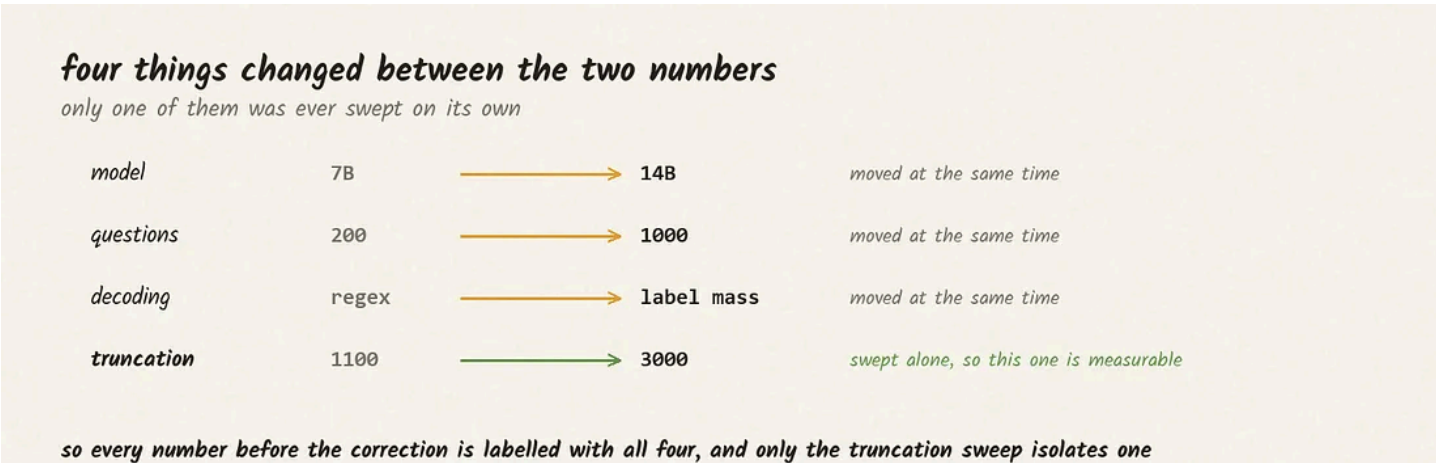
The agent stays in the repository as a measured ablation rather than the headline. Reporting it as the architecture would describe a diagram rather than a result.



The Measurement Everything Else Was Standing In For

Every result so far concerns groundedness, coverage and refusal. **None of them show the answers are more often correct than ordinary vector retrieval.**

One warning covers this section and the four after it. Everything up to the correction ran through a harness that truncated every document to **1100 characters**, on two hundred questions, with the label parsed from free text by a regular expression. All three matter.



Four axes moved at once, and only one of them was ever swept alone (Created by [Fareed Khan](#))

Corpus, questions, generator and prompt are fixed, so the only variable is how evidence is found.

```
@torch.no_grad()
def vector_search(question, k=TOPK):
    qv = torch.from_numpy(RK.embed_query(question)).cuda().half()
    top = torch.topk(V @ qv, k)
    return [int(idx_pmids[i]) for i in top.indices.cpu().numpy()], top.values.to(device)
```

Beating a weak baseline proves nothing, so Arm A got 20.3 GiB of VRAM and an exhaustive scan.

selective = coverage × accuracy



Value per question asked rather than per question answered, so refusing everything scores zero

Selective accuracy, the one number that cannot be gamed by answering only the easy questions (Created by [Fareed Khan](#))

OUTPUT

arm	coverage	accuracy	selective
A vector RAG (exact, 28.3M)	100.0%	48.0%	48.0%
B graph (no gate)	67.0%	30.6%	20.5%
C graph + refusal gate	59.0%	33.9%	20.0%

label distribution {'maybe': 28, 'no': 70, 'yes': 102}

MAJORITY-CLASS BASELINE always answer 'yes' -> 51.0%

A vector RAG (exact, 28.3M)	48.0%	does NOT beat the constant
B graph (no gate)	30.6%	does NOT beat the constant
C graph + refusal gate	33.9%	does NOT beat the constant

Selective accuracy is 48.0% for vector retrieval and 20.0% for the gated graph, and neither beats a constant. Always answering yes scores 51.0%.

Both arms losing to a constant is not evidence that one beats the other. It is evidence that **this benchmark plus this generator cannot separate them**, and I should have asked why.

$$\text{floor} = \max_c \frac{|\{i: y_i = c\}|}{N}$$

The constant predictor every result has to clear, which is 53.3 percent on the 600 held-out questions

The constant predictor every result has to clear (Created by [Fareed Khan](#))

The cause of the loss is measurable.

OUTPUT

gold PMID retrieved in top-8

vector RAG 95.5%

graph 15.0%

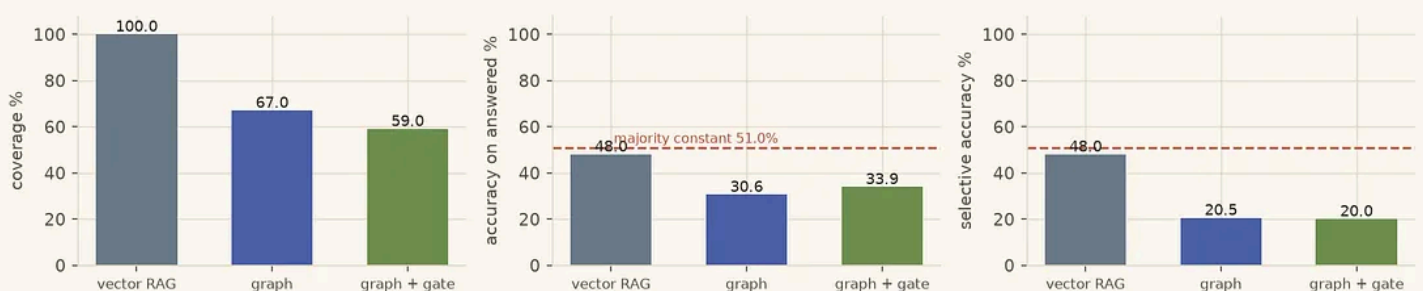
graph, when it answers 22.9%



Dense retrieval puts the gold document in the top eight on 95.5% of questions and the graph manages 15.0%. That is either a bug or a limit, and those need opposite responses, so I decomposed it.

Graph retrieval against exact dense search, and neither beats a constant

7B generator, n=200, free-text regex, 1100-character evidence. Gold reached the top 8 for 95.5% of dense queries and 15.0% of graph queries.



Graph retrieval against exact dense search, both below the floor (Created by [Fareed Khan](#))

Diagnosing the Retrieval Gap

Three causes, each with a different fix. The gold article might carry none of the grounded concepts, might rank below the top eight, or might never enter the candidate set. Those are grounding's fault, the ranker's fault, and my own bug.

So I checked membership at each stage.

```
full = set(int(x) for x in both)
if gi not in full:
    stats["C1 gold not co-annotated"] += 1
    continue

trunc = set(int(x) for x in both[-CL.max_paths:])
if gi not in trunc:
    stats["C3 TRUNCATED AWAY"] += 1
    continue

order = [S.idx(p.terminal_pmid) for _, p in RK.rank(q, usable, top_k=len(usable))]
if gi in order:
    stats["HIT top-8" if order.index(gi) < TOPK else "C2 ranked below top-8"]
```

OUTPUT

where the gold document is lost	count	share
C1 gold not co-annotated	116	58.0%
C0 fewer than 2 concepts	31	15.5%
HIT top-8	25	12.5%
C3 TRUNCATED AWAY	23	11.5%
C2/C3 gold only via bridge	4	2.0%
C2 ranked below top-8	1	0.5%

candidate-set size: median 49 p90 3,257 max 103,835
max_paths cap was 120 -> 40.8% of questions lost
candidates before the ranker ever saw them

when gold IS a candidate: median rank 0, top-8 96%, top-30 100%

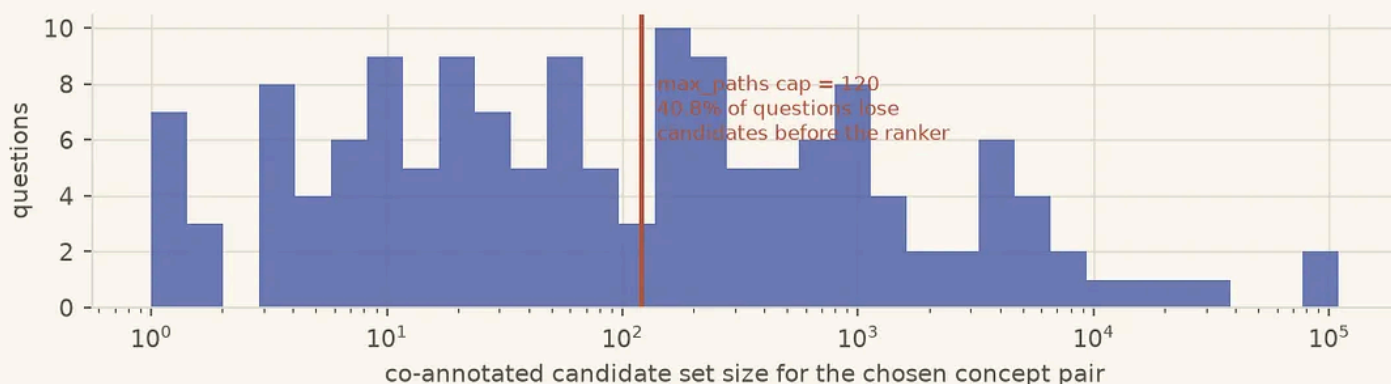
Median rank zero, so **the ranker was never the problem**. Gold that reaches the candidate set lands 96% inside the top eight, 100% inside the top thirty.

My cap of 120 discarded candidates on 40.8% of questions, against a median set of 49 and a ninetieth percentile of 3,257. **The cap fell in the middle of the distribution.**



The cap fell in the middle of the distribution

median 49, p90 3,257, max 103,835. The ranker never saw the discarded candidates.



The cap fell in the middle of the distribution (Created by [Fareed Khan](#))



The second suspect, `major_only`, keeps 32.7% of MeSH edges and **discards 67.3% of the entity layer**. I chose it for precision, but a document I never retrieve cannot be ranked, cited or refused.

The arms are cumulative so the interaction is visible.

OUTPUT

arm	gold in set	gold top-8
A major_only=True, cap=120 (current)	13.0%	12.5%
B major_only=True, no cap	24.5%	21.5%
C major_only=False, cap=120	9.0%	9.0%
D major_only=False, no cap (both fixes)	33.0%	26.5%

vector RAG reference: 95.5%

Minor topics alone made things worse, 12.5% down to 9.0%, because a larger candidate set means the cap discards more of it. Only with the cap removed do they help, 21.5% to 26.5%.

Independent ablations would have concluded that minor topics hurt, dropping a fix worth five points.

Three quarters of the retrieval gap is structural

n=200, 1100-character era. Including minor topics ALONE made things worse, because a larger candidate set means the 120 cap discards more of it.



Three quarters of the gap is structural, one quarter was mine (Created by [Fareed Khan](#))

And 26.5% against 95.5% is not a bug that remains, it is a limit. MeSH assigns roughly twelve headings from a 31,110-term vocabulary, so two headings identify a topic and not a document. Dense embeddings encode full text and single out one paper. Annotation graphs cannot.



The conclusion is not that graphs do not work, but that I had given the graph the wrong job.

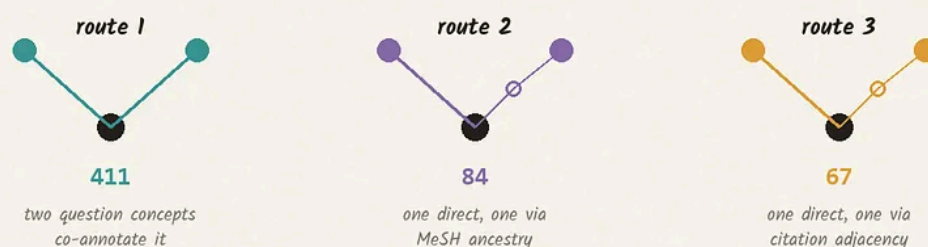
Dense Retrieves, the Graph Certifies

Retrieval is a recall problem and dense vectors win it. What they cannot do is say **why** a document is admissible evidence, or refuse when none is.

If certification rejected gold as often as it rejected noise, the graph would add nothing.

three ways the graph will vouch for a document

and none of them predicts whether the answer is right



AUROC 0.500. Certified questions are 82.2% right and refused ones 82.5% right.

it is declining questions the system would have got right, chosen at random

The three routes, verbatim from the rule the thesis rests on (Created by [Fareed Khan](#))

```
def certify(pmid, concept_ds):
    gi = S.idx(pmid)
    if gi < 0:
        return False, "not_in_graph"
    ds_ = set(int(x) for x in S.a2m_indices[S.a2m_indptr[gi]:S.a2m_indptr[gi + 1]])

    hit = ds_ & set(concept_ds)
    if len(hit) >= 2:
        return True, f"co-annotated with {len(hit)} question concepts"
    if len(hit) == 1:
        for d in set(concept_ds) - hit:
            anc = {S.mesh_idx(a) for a in TREE.ancestors(str(S.mesh_uis[d]), 2)}
            if ds_ & anc:
                return True, "one concept direct + one via MeSH ancestry"
        # Route 3 repeats this over citation neighbours, capped at 200.
        return False, "only one question concept"
    return False, "no question concept"
```



OUTPUT

certification outcome	count
no question concept	545
only one question concept	519
co-annotated with 2 question concepts	277
one concept direct + one via citation adjacency	74
co-annotated with 3 question concepts	73
one concept direct + one via MeSH ancestry	62
no_concepts	40
co-annotated with 4 question concepts	10
gold retrieved by dense	95.5%
... and certified by the graph	47.1%
non-gold certification rate	28.8%

Gold documents certify at 47.1% and non-gold at 28.8%, so the signal is real.

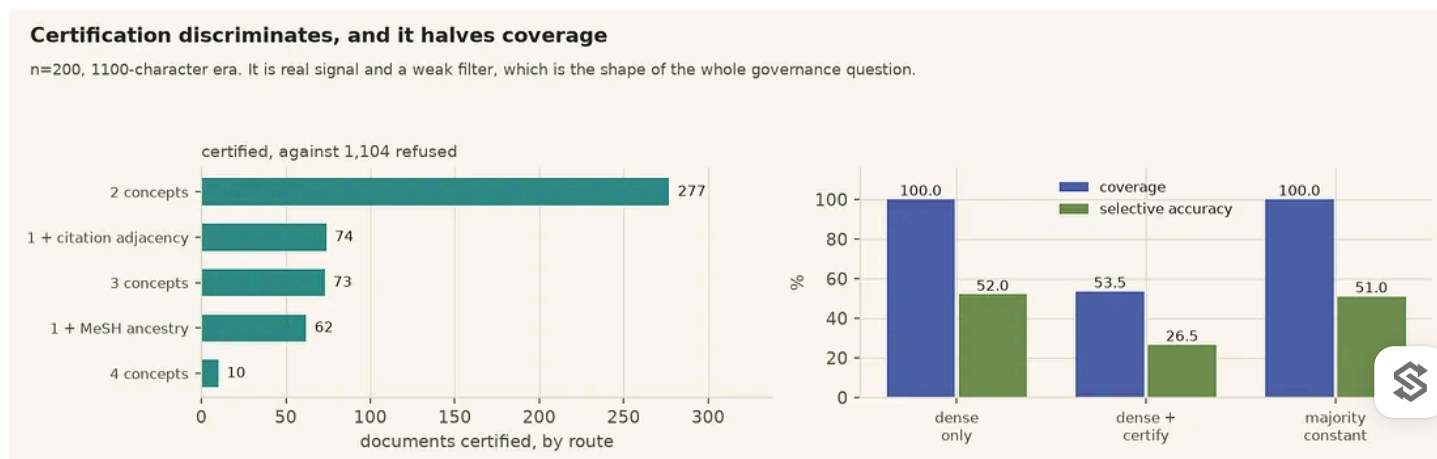
Certifying fewer than half the correct documents is a weak filter, and the cost shows immediately.

OUTPUT

arm	coverage	accuracy	selective

A dense only	100.0%	52.0%	52.0%
E dense + graph certification	53.5%	49.5%	26.5%
majority-class constant	100.0%	51.0%	51.0%

Coverage falls from 100.0% to 53.5% and selective accuracy halves with it. On the 1100-character harness I had tested graph retrieval three ways and it lost three times.



Certification discriminates, and it halves coverage (Created by Fareed Khan)

Was the Measurement Instrument Broken?

Every accuracy number above came from one seven-billion-parameter generator, and the best arm scored 52.0% against a 51.0% constant. **That is close enough to chance that the instrument may not have resolved the question at all.**

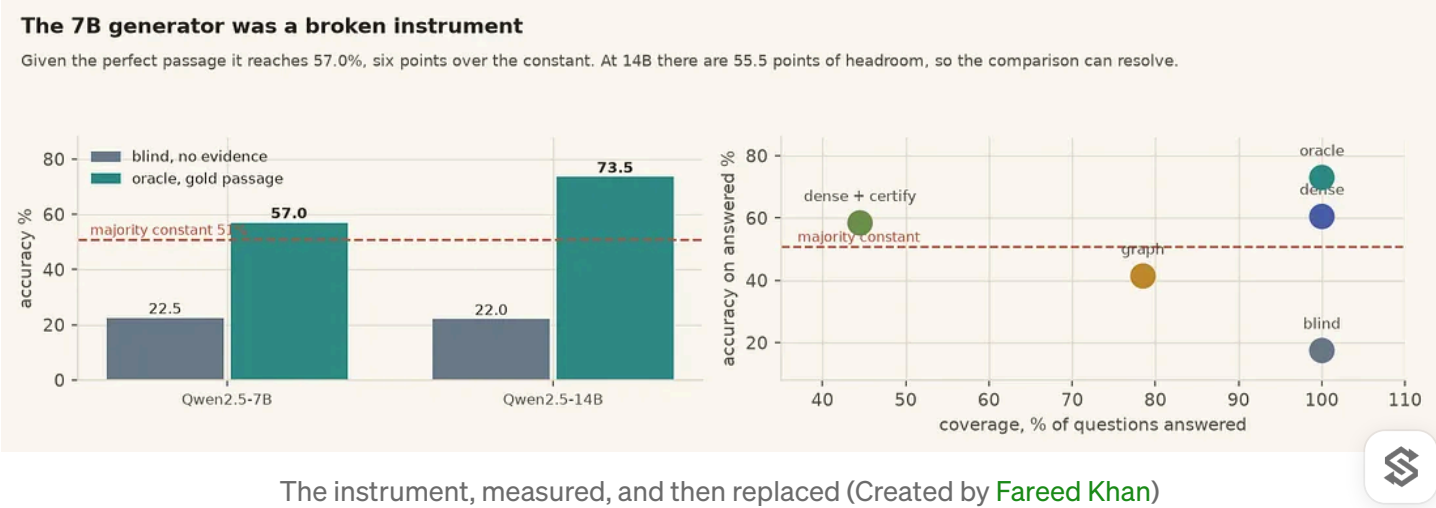
The oracle arm settles it. No retriever beats handing over the gold passage, so oracle accuracy bounds every architecture from above and a blind arm bounds it from below.

OUTPUT

model	blind	oracle	lift	vs const
Qwen2.5-7B-Instruct	22.5%	57.0%	+34.5	+6.0
Qwen2.5-14B-Instruct	22.0%	73.5%	+51.5	+22.5
majority constant: 51.0%				

Given the perfect passage the seven-billion model reaches 57.0%, six points above the constant. Dense retrieval scored 52.0%, about ninety percent of that ceiling, so the whole comparison lived in five points of headroom.

The fourteen-billion model reaches 73.5%, 22.5 points above the constant, and blind accuracy stays near 22% at both sizes, so the models need the evidence. A **ceiling arm belongs in every retrieval evaluation**, because without it our method scored 52% is uninterpretable.



The Distractor Gap

Re-running with headroom produced a number that should have stopped me earlier.

```
#### OUTPUT ####
n = 200  oracle 75.0%  blind 29.0%  majority 51.0%
```

k	dense (bi-encoder)	cross-encoder rerank	graph rerank (H-C)
1	61.5%	64.5%	55.0%
2	61.0%	63.0%	54.0%
3	62.5%	63.5%	53.0%
5	63.0%	62.5%	61.5%
8	62.5%	64.5%	64.5%
12	63.5%	63.5%	64.0%

The oracle, given the gold document **alone**, scores 75.0%. Dense retrieval puts that same document in the pool 98% of the time and scores 62.5% with it plus seven neighbours.

The model performs worse with the right document plus seven others than with the right document by itself. I had been trying to retrieve better while retrieval was already succeeding.

Labels now come from **constrained decoding**, reading probability mass over the three label tokens, so the free-text regular expression leaves the loop.

$$P(y) \propto p(t_y) + p(T_y), \quad y \in \{\text{yes, no, maybe}\}$$

The label is read as probability mass over the lower-case and capitalised label tokens, so no regex is in the loop

The label read as probability mass, not parsed out of prose (Created by [Fareed Khan](#))

```
@torch.no_grad()
def decide(user):
    msgs = [{"role": "system", "content": PROMPT}, {"role": "user", "content": user}]
    enc = tok.apply_chat_template(msgs, add_generation_prompt=True,
                                  return_tensors="pt", return_dict=True).to("cuda")
    p = torch.softmax(gen(**enc).logits[0, -1].float(), -1)
    s = np.array([float(p[a] + p[b]) for a, b in zip(LAB_IDS, LAB_CAP)])
    return LABELS[int(s.argmax())], float(s.max() / max(s.sum(), 1e-9))
```

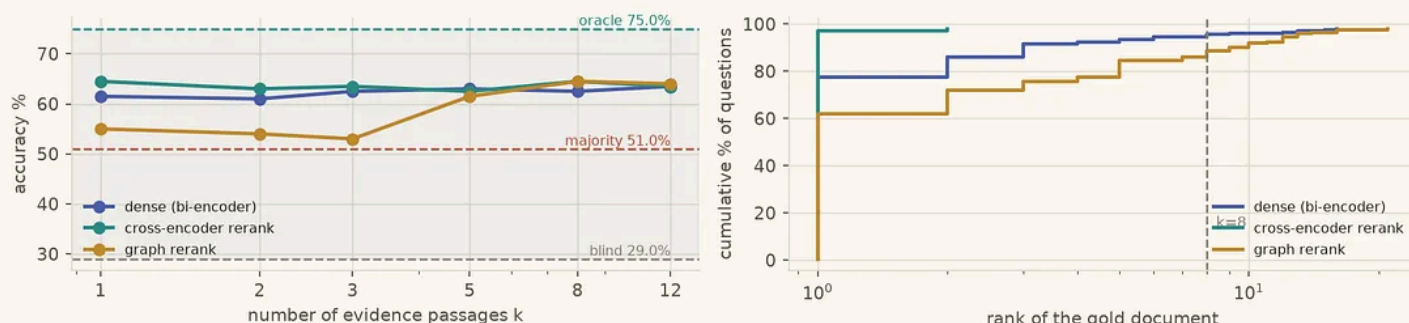
Summing the lower-case and capitalised token per label matters, because a model that writes Yes rather than yes is differently formatted, not less certain.

```
#### OUTPUT ####
dense (bi-encoder)      in pool 98.0% rank-1 79.1% top-3 93.4% median
cross-encoder rerank   in pool 98.0% rank-1 99.0% top-3 100.0% median
graph rerank (H-C)     in pool 98.0% rank-1 63.3% top-3 77.0% median
```

A cross-encoder puts the gold document at rank one 99% of the time. Ranking is finished, and the remaining gap is about reading rather than retrieval.

The right document is already there, and every neighbour is a risk

n=200, 1100-character era. Oracle 75.0% against the best retrieved cell 64.5%, with gold in the pool 98%.



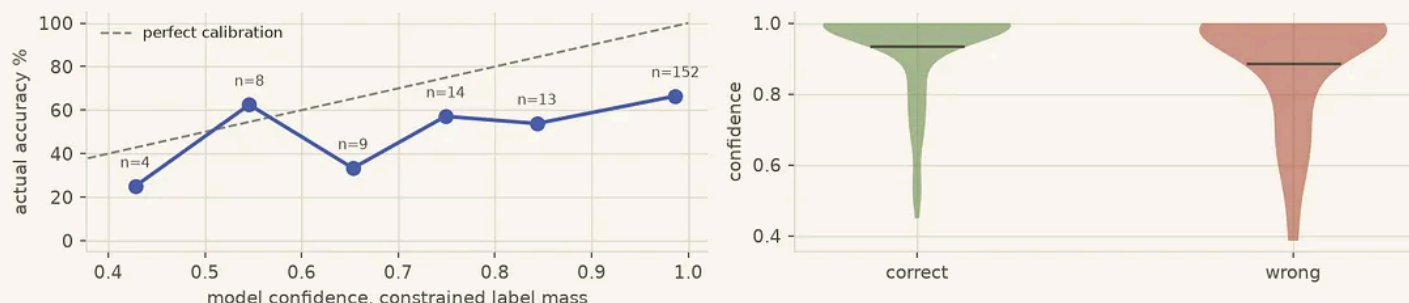
Every extra passage is a chance to mislead (Created by [Fareed Khan](#))

The model's confidence already predicts whether it is right, which returns at the end of this post.



The model already knows when it is about to be wrong

This is the signal that later beats graph certification as an abstention rule, and it costs nothing to compute.



The model already knows when it is about to be wrong (Created by [Fareed Khan](#))

The Constant That Was Worth Fourteen Points

I had spent a great deal of effort closing that gap by ranking better. Cross-encoders, rank fusion, marginalisation, prompt variants, calibration, and **none of it beat a plain cross-encoder top-one prompt.**

When a dozen reasonable interventions produce nothing, the thing being measured is usually not what you think it is. Every experiment truncated retrieved documents to **1100 characters**, a constant I never examined.

The oracle arm received 4000, so the distraction gap was never one document against eight, it was a complete abstract against eight truncated ones.

A PubMed abstract is structured: background, methods, results, **conclusions**. The conclusion answers the yes or no question, and a cut at 1100 characters removes it.

First I measured how much text was being cut.

```
top1 = [int(pool[i][ORDCE[i][0]]) for i in range(N)]
L = np.array([len(ABST.get(p, "")) for p in top1])
for t in (1100, 2000, 3000, 4000):
    print(f" truncated at {t:>5}           {100*np.mean(L>t):5.1f}% of documents lost
          f" (mean loss {np.maximum(L-t,0).mean():.0f} chars)")
```

OUTPUT

Length of the document the system actually reads (cross-encoder rank 1), measured over all 1000 expert-labelled questions:

mean 1718 chars	median 1720	p90 2187	max 3224
-----------------	-------------	----------	----------

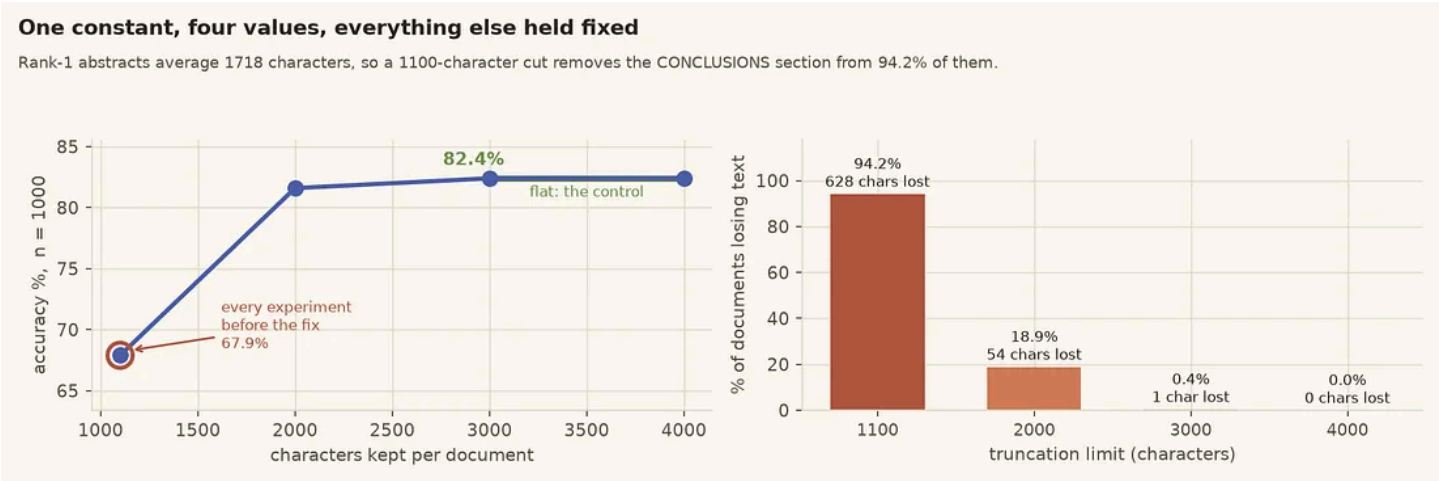
truncation	documents losing text	mean chars lost	
1100	94.2%	628	<-- every experiment I
2000	18.9%	54	
3000	0.4%	1	
4000	0.0%	0	

At 1100 characters, 94.2% of documents lose text, and the median abstract loses its final 620 characters. That is the sentence saying whether the finding was positive, negative or mixed.

Now the sweep. One constant, four values, everything else fixed: same thousand questions, same pools, same ordering, same generator.

```
#### OUTPUT ####
configuration      accuracy  delta vs 1100
-----
k=1, trunc 1100    67.9%      +0.0
k=1, trunc 2000    81.6%      +13.7
k=1, trunc 3000    82.4%      +14.5
k=1, trunc 4000    82.4%      +14.5
k=8, trunc 3000    80.3%      +12.4
```

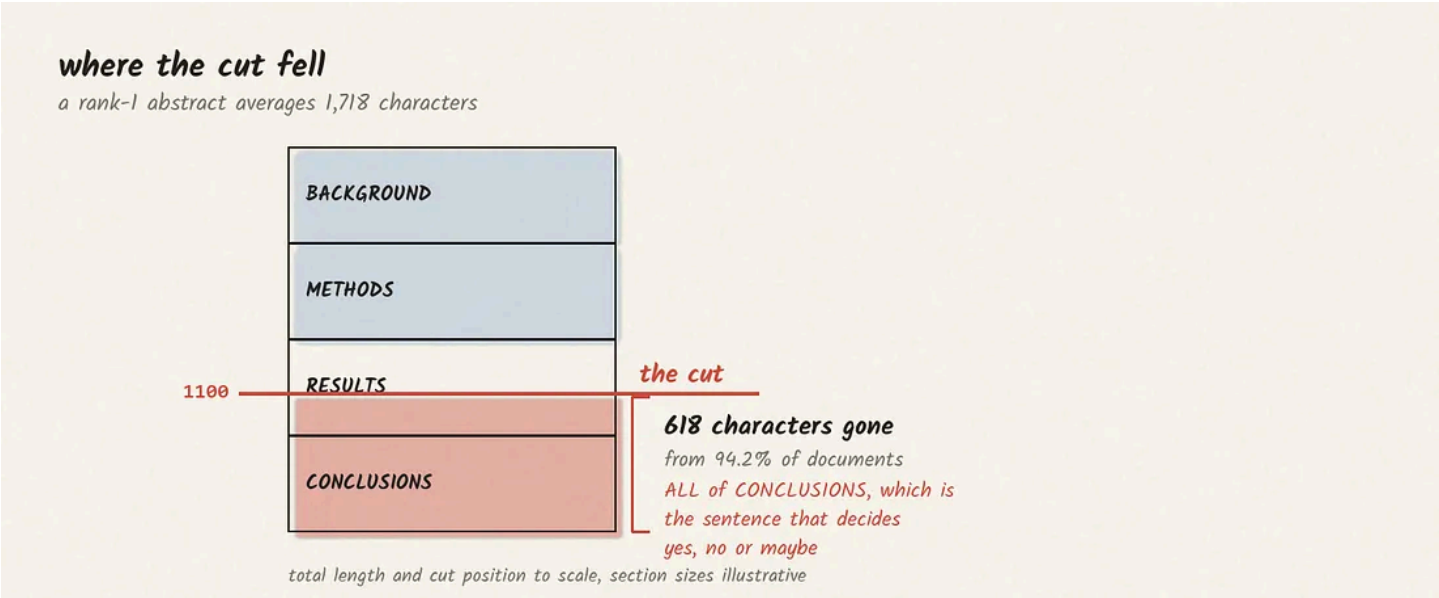
Raising one constant from 1100 to 3000 moved accuracy by 14.5 points. The flat run from 3000 to 4000 is the control, because 99.6% of abstracts are complete by then, so the effect is truncation.



One constant, four values, everything else held fixed (Created by [Fareed Khan](#))



It is the only comparison in the project with zero confounds, which is why I trust it.



Where the cut fell, and what it removed (Created by [Fareed Khan](#))

The Rerun, and the Discipline Around It

Regenerating the posterior cache made everything after it cheap. Sixteen candidates per question are scored **alone**, producing a full distribution over the three labels, so every decision rule, mixture weighting, k and abstention policy becomes arithmetic.

$$P(y) = \sum_{i=1}^k w_i P(y \mid q, d_i), \quad \sum_i w_i = 1$$

Each document is scored alone and the posteriors mixed afterwards, so a distractor can dilute but never derail

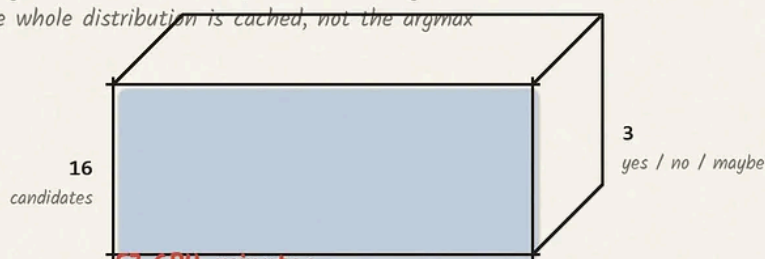
Each document scored alone, then the posteriors mixed afterwards (Created by [Fareed Khan](#))

Two details make this pass about five times faster across forty six thousand forward passes. Asking for one logit position avoids materialising 4.0 GiB at batch 4, and length sorting stops a 200-token abstract paying for a 3000-token neighbour.

```
@torch.no_grad()
def posteriors(users, sys_prompt, bs):
    """Batched constrained decoding: full normalised P over {yes,no,maybe}."""
    txt = [tok.apply_chat_template(
        [{"role": "system", "content": sys_prompt}, {"role": "user", "content":
        add_generation_prompt=True, tokenize=False) for u in users]
    order = np.argsort([len(t) for t in txt], kind="stable")
    out = np.zeros((len(users), 3), dtype=np.float32)
    for s in range(0, len(order), bs):
        sel = order[s:s + bs]
        e = tok([txt[i] for i in sel], return_tensors="pt", padding=True,
            truncation=True, max_length=8000,
            add_special_tokens=False).to("cuda")
        p = torch.softmax(gen(*e, logits_to_keep=1).logits[:, -1].float(), -1)
        sc = torch.stack([p[:, a] + p[:, b] for a, b in zip(LAB_IDS, LAB_CAP)]),
        out[sel] = (sc / sc.sum(1, keepdim=True).clamp_min(1e-9)).cpu().numpy()
    return out
```

pay the GPU once, then every decision is arithmetic

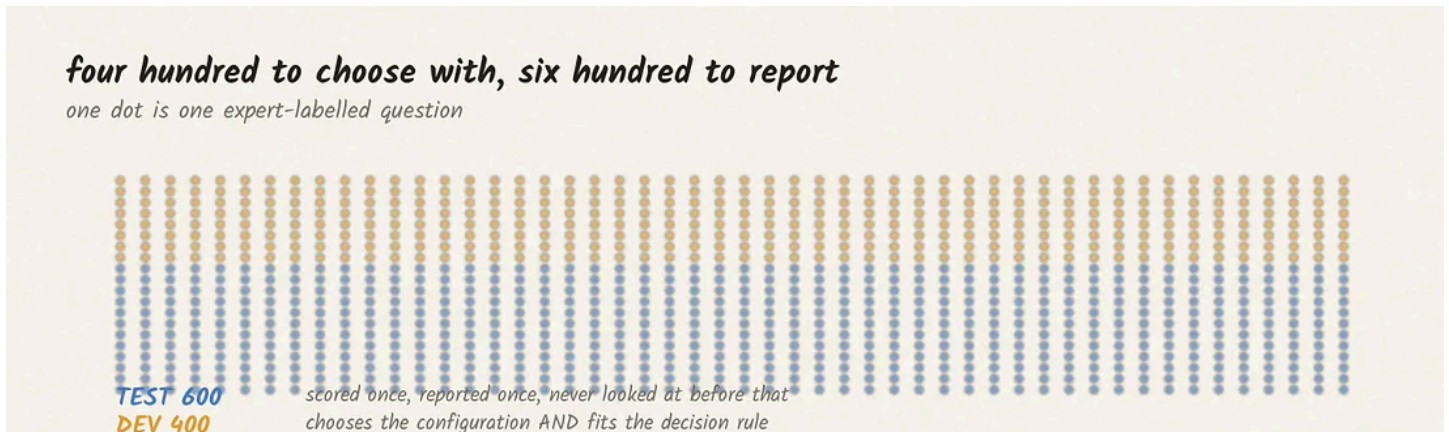
the whole distribution is cached, not the argmax



1000 questions

then 139 configurations, the calibration fit, the error budget and the governance test are all numpy

Because the cache makes another configuration free, it is easy to fool yourself, so the split is enforced and never crossed.



Four hundred to choose, six hundred to report, and they never meet (Created by [Fareed Khan](#))



```
rng = np.random.default_rng(0)
perm = rng.permutation(N)
DEV, TEST = np.sort(perm[:400]), np.sort(perm[400:])

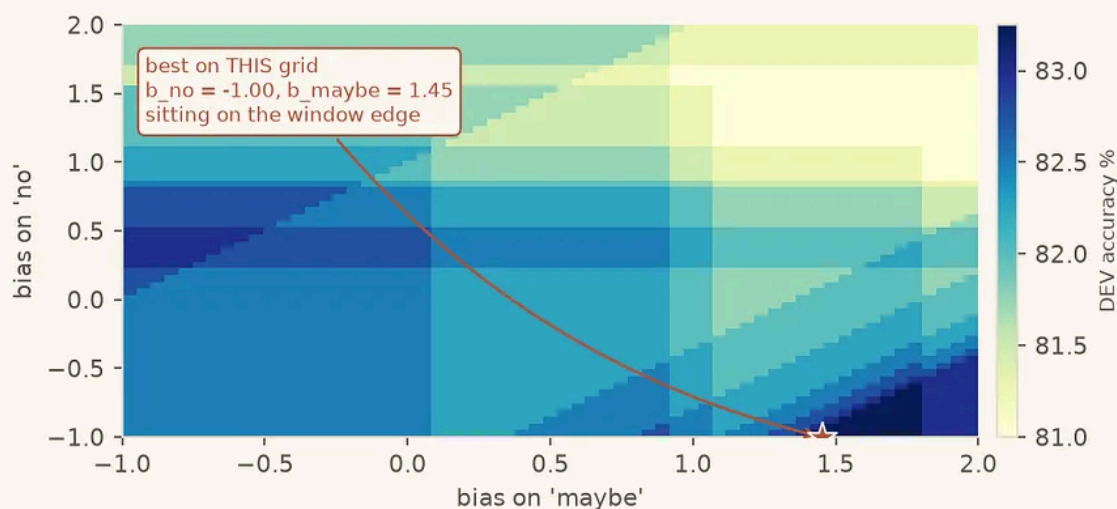
# The oracle and blind arms are REFERENCE POINTS, not deployable systems. Left
# in the pool once, this sweep picked oracle+calibration at 77.2%, a number
# that cannot be produced at inference time.
SELECTABLE = np.array([i for i, n in enumerate(NAMES)
                        if "oracle" not in n and "blind" not in n])
```

The class bias is fitted by grid search, and my first grid was too narrow.

```
def fit_bias(p, idx, grid=np.arange(-3.0, 3.01, 0.05)):
    # An earlier run used [-1, 2] and the optimum landed exactly on the -1.00
    # edge for both classes, so the reported bias was an artefact of the grid.
```

The search window that was too narrow

The optimum sits on the edge of the window, which means the grid clipped it. Widening the search moved the fitted bias to -2.40.



The optimum sitting on the edge of the window, which is how you know the window is wrong (Created by [Fareed Khan](#))

An optimum that lands exactly on the boundary of your search is not an optimum, it is a clipped search.

The Corrected Result

Four hundred development questions choose the configuration and fit the decision rule. Six hundred held-out questions are scored once.

$$\hat{y} = \arg \max_c [\log P(c | q, D) + b_c], \quad b_{\text{yes}} = 0$$

One additive log-space bias vector, fitted on the 400 development questions and never on the test set

The decision rule, one additive bias vector fitted on development data only (Created by [Fareed Khan](#))

OUTPUT

system	TEST	95% CI
blind (no evidence at all)	37.2%	
majority class (constant 'yes')	53.3%	

PREVIOUS BEST, trunc 1100	68.8%	[65.0, 72.5]
corrected, argmax	82.3%	[79.3, 85.3]
corrected + DEV-fitted calibration	83.2%	[80.2, 86.2]

Winner selected on DEV: p0|concat|ce_k1
 configurations searched: 139 DEV 400 TEST 600
 DEV->TEST correlation across all configs: 0.990

83.2% on six hundred held-out questions, with a 95% interval of 80.2 to 86.2, against a 53.3% majority floor and a 37.2% no-evidence floor.

$$CI_{95} = [Q_{2.5}(\bar{a}^{(b)}), Q_{97.5}(\bar{a}^{(b)})], \quad b = 1 \dots 4000$$



Paired bootstrap over the same held-out questions, resampled four thousand times

Paired bootstrap over the same held-out questions (Created by [Fareed Khan](#))

$$p = 2 \cdot 2^{-n} \sum_{i=0}^{\min(n_{01}, n_{10})} \binom{n}{i}, \quad n = n_{01} + n_{10}$$

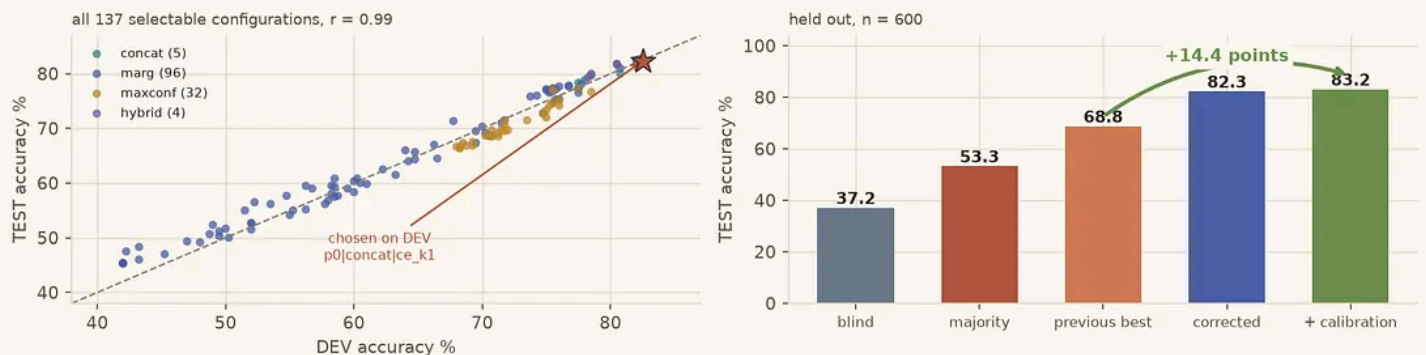
Exact paired test, where only the questions the two systems disagree on carry any information

Exact McNemar, where only the disagreements carry information (Created by [Fareed Khan](#))

The winner is a plain cross-encoder top-one prompt, chosen over marginalisation, rank fusion, hybrid mixtures and prompt variants.

The corrected result

Configuration and decision rule fitted on 400 DEV questions, reported once on 600 held-out TEST questions.



The corrected result, and every configuration that lost (Created by [Fareed Khan](#))

Two checks belong beside that number. The first is what picking a winner is worth on noise.



OUTPUT

Labels shuffled, so no configuration can carry real signal.

best of 137 47.5%

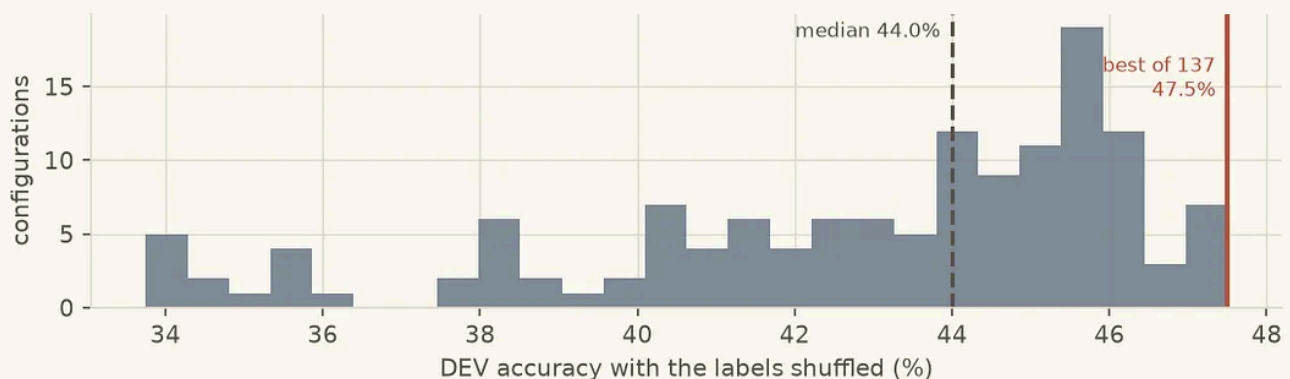
median 44.0%

spread max - median 3.5 points

With the labels shuffled, the best of 137 configurations still clears the median by 3.5 points. That is why the development and test split is not optional.

How much a winner is worth before it means anything

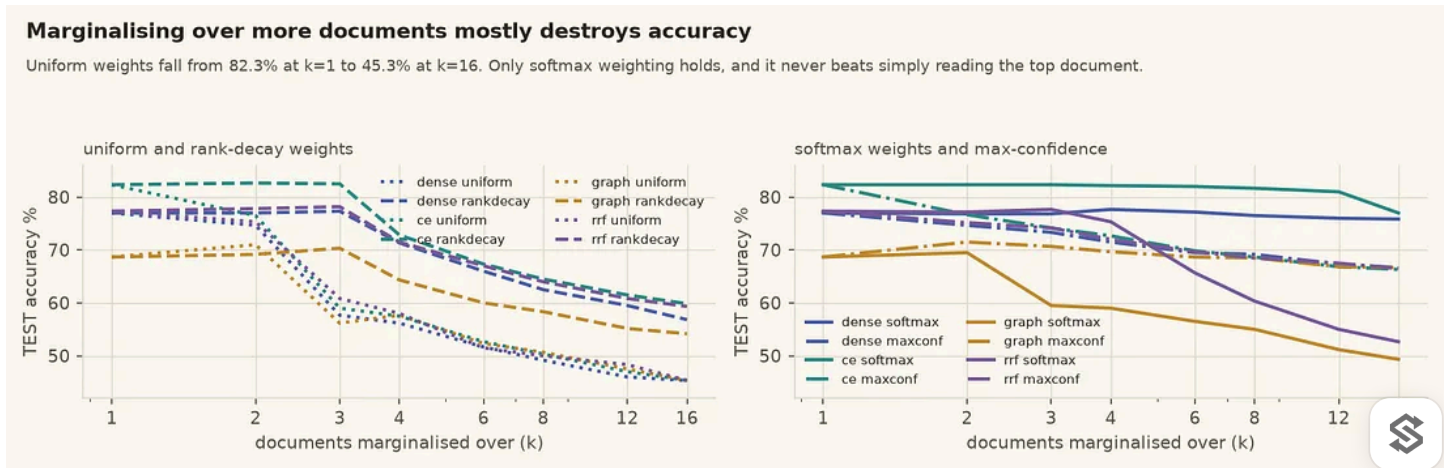
With shuffled labels no configuration can carry signal, yet the best of 137 still clears the median by 3.5 points.



What picking a winner is worth before it means anything (Created by [Fareed Khan](#))

The second is that the best score on test is not the reported score. The best configuration on test reaches 82.7% under argmax against the chosen arm's 82.3%, and reporting 82.7% means reporting the maximum of 137 noisy estimates.

Here is what marginalisation actually did.



Marginalising over more documents mostly destroys accuracy (Created by [Fareed Khan](#))

Uniform weighting falls from 82.3% at one document to 45.3% at sixteen, and only softmax holds up, never beating the top document alone. **The alternatives diluted a good answer with worse ones.**

$$w_i^{\text{unif}} \propto 1, \quad w_i^{\text{decay}} \propto \frac{1}{1+i}, \quad w_i^{\text{soft}} \propto e^{z_i}$$

Three mixture weightings over the top-k, where z is the ordering score standardised within the question

The three mixture weightings that were swept (Created by [Fareed Khan](#))

$$\text{rrf}(d) = \sum_{o \in \mathcal{O}} \frac{1}{60 + \text{pos}_o(d)}$$

Reciprocal rank fusion is rank-based, which matters because the three scores are a cosine, a logit and a count

Reciprocal rank fusion, which needs no score normalisation (Created by [Fareed Khan](#))

$$\hat{y} = \arg \max_y P(y \mid q, d_{i^*}), \quad i^* = \arg \max_i \max_y P(y \mid q, d_i)$$

Keep only the single document the reader is most certain about, and let that document decide

Max-confidence, letting the reader pick its own single document (Created by [Fareed Khan](#))

Three Conclusions That Did Not Survive

A measurement error makes results **wrong in a direction**, and the direction favoured the story I was telling.

OUTPUT

(a) THE DISTRACTION GAP

previously reported 12.7% of error was distraction
corrected 2.0%

-> 10.7 of those 12.7 points were the oracle arm
reading 4000 characters **while** the deployed arm **read** 1100.

(b) 'maybe' IS UNDETECTABLE

at trunc 1100 AUROC **for** detecting gold='maybe' 0.548 (chance)
corrected 0.755

-> The hedging language that signals an inconclusive study lives **in**
the CONCLUSIONS sentence. We were deleting the evidence **for** the
class and **then** concluding the model could not represent it.

(c) THE BIGGER MODEL IS WORSE

	14B	32B
at trunc 1100	67.9%	55.1%
corrected	82.4%	82.4%

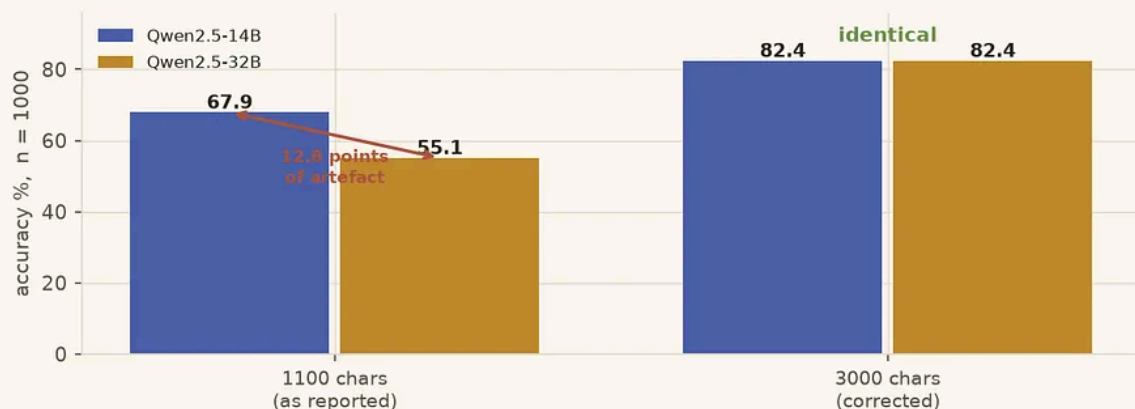
Eighty four percent of the distraction gap was my harness. **It was the oracle reading complete abstracts while the deployed arm read truncated ones.**

The maybe class was never undetectable. The hedging that signals an inconclusive study lives in the conclusions sentence, so I deleted the evidence for a class, then concluded the model could not represent it.

The larger model was not worse. It answered maybe 34% of the time on truncated abstracts and was scored wrong, but with the conclusion removed, maybe was the correct reading.

Model scale, measured twice

On truncated abstracts the larger model answered 'maybe' far more often and was marked wrong for it, which was the correct reading of what it was given.



Model scale, measured twice (Created by Fareed Khan)

The notebook does not make the caveat that narrows that claim. Fitting the class bias **per model** on the same split moves the thirty two billion model from 53.7% to 66.5% at 1100 characters and the fourteen billion from 68.8% to 70.0%, a thirteen point gap shrunk to three and a half without touching truncation.

So the statement is narrower than the one I first wrote. Comparing two models by argmax compares their thresholds as well as their knowledge, so most of the apparent scale effect was threshold and truncation explains the rest.

Comparing two models by argmax compares their thresholds too

At 1100 characters the gap on the deployed arm is 15.2 points under argmax and 3.5 once each model gets its own fitted bias.



Comparing two models by argmax compares their thresholds too (Created by Fareed Khan)

The Corrected Error Budget

Every error on the six hundred held-out questions, assigned to exactly one cause.

$$1 = \underset{0.823}{\text{correct}} + \underset{0.013}{\text{retrieval}} + \underset{0.020}{\text{distraction}} + \underset{0.143}{\text{residual}}$$

Every error on the 600 held-out questions assigned to exactly one cause, which says what to work on next

Every error assigned to exactly one cause (Created by [Fareed Khan](#))

OUTPUT

bucket	n	share	what would fix it
answered correctly	494	82.3%	-
gold never retrieved	8	1.3%	better retrieval
distraction	12	2.0%	better ranking / reading
both renderings wrong	86	14.3%	reading or label limit

gold	n	correct	retrieval	distract	residual
yes	320	90.0%	1.2%	2.5%	6.2%
no	218	94.5%	0.9%	1.8%	2.8%
maybe	62	0.0%	3.2%	0.0%	96.8%

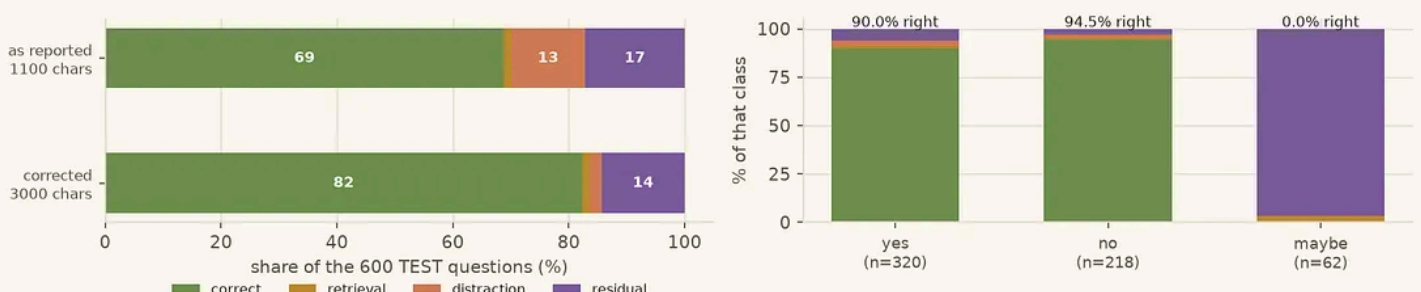


Retrieval is effectively solved at 1.3%. A 929-million-edge graph and a dense index over 28.3 million abstracts put the right document in a sixteen-candidate pool 97.4% of the time, and distraction, my central problem, is 2.0%.

Yes and no are at 90.0% and 94.5%, and **the entire remaining error budget is the maybe class**, 10.3% of the benchmark and one on which argmax scores zero.

The correction moved the error, it did not just move the score

Distraction fell from 12.7% to 2.0% and retrieval never moved at 1.3%. Everything left sits in one class.



The retrieval bucket is 1.33% in both the old and corrected measurement, the internal control proving the correction touched reading and nothing else.

One caveat inverts something I said earlier. **The oracle arm is no longer an upper bound.** It read four thousand characters and was never re-run, so at the corrected truncation the deployed arm scores 82.3% and the oracle scores 76.7%. That bucket is not capacity, it is where both renderings fail, so any sentence about capturing a fraction of the headroom belongs to the old harness.

Every arm at the corrected truncation, against the floor

Reading the single best document beats every fusion arm, and it also beats the arm that was supposed to be the ceiling.



Every arm at the corrected truncation, against the floor (Created by Fareed Khan)

Is the maybe class reachable? The posterior ranks those questions at AUROC 0.755, so the information is present and the decision rule discards it. Calibration takes recall from zero to five of sixty two.

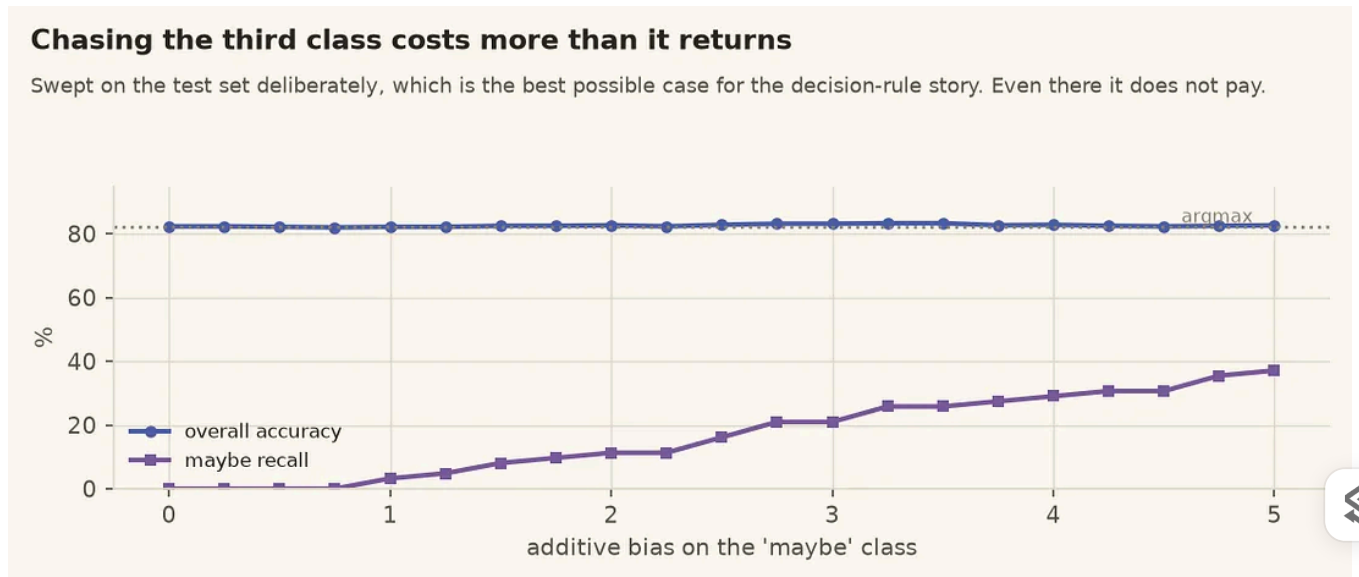
What the calibration actually bought

It recovers five of the sixty-two 'maybe' questions and costs a little accuracy on 'no'. The class is reachable, but not profitably.

		argmax			maybe recall 0.0%	
gold	yes	288	30	2		
	no	11	206	1		
	maybe	31	31	0		
		yes	no	maybe		
		predicted				

		DEV-calibrated			maybe recall 8.1%	
gold	yes	293	22	5		
	no	11	201	6		
	maybe	35	22	5		
		yes	no	maybe		
		predicted				

But recovering it costs more yes and no accuracy than it buys. I swept the bias on the test set deliberately, the best case for that story, and it does not pay.



Chasing the third class costs more than it returns (Created by Fareed Khan)

That is a real property of the benchmark rather than a bug this time, which matters because everything else here was a bug.

Re-examining the Governance Claim

The claim this project was built on is that graph certification does not buy accuracy, it buys the **right to refuse**. It was measured at 1100 characters on two hundred questions, and needs no GPU, so it recomputes on all thousand.

An abstention policy is only worth its complexity if it beats the free alternative, which here is the model's own confidence.

OUTPUT

Matched on coverage - both policies answer the same NUMBER of questions, so only the CHOICE of which ones differs.

coverage	graph certification	model confidence	edge
90%	82.3%	87.6%	-5.3
80%	82.4%	90.4%	-8.0
70%	82.4%	93.3%	-10.9
60%	82.4%	93.6%	-11.2

50%	82.4%	94.0%	-11.6
40%	83.8%	95.0%	-11.2

Graph certification is flat at 82.4%. Refusing half the questions leaves accuracy at the unfiltered rate, while confidence climbs from 82.3% to 94.0%.

That used concept overlap, the crudest of three routes. Here is the full rule.

OUTPUT

Which route earned the certification?

no_question_concept	438	(43.8%)	
route1_coannotated	411	(41.1%)	
route2_mesh_ancestry	84	(8.4%)	<- graph-specific
route3_citation_adjacency	67	(6.7%)	<- graph-specific

The two graph-specific routes account **for** 15.1% of decisions.
certified within top-8 56.2% of questions

signal	AUROC
-----	-----
path certification, rank-1 doc	0.500
path certification, best of top-8	0.497
concept overlap only	0.519
model confidence max P(y) [FREE]	0.810 <- free
accuracy on CERTIFIED questions	82.2%
accuracy on REFUSED questions	82.5%
difference	-0.4 points

Path certification predicts correctness at AUROC 0.500, chance to three decimal places. Certified questions are correct 82.2% of the time and refused questions 82.5%, so it declines questions it would have got right, at random. The two graph-specific routes fired on 15.1% of decisions and moved it from 0.519 to 0.497.

$$\text{AUROC} = \frac{\sum_{i \in \mathcal{P}} r_i - \frac{n_1(n_1 + 1)}{2}}{n_1 n_0}$$

Rank-based, with tied scores given their mean rank, which is the whole difference between 0.539 and 0.519 here



The graph does not know which questions it will get right (Created by [Fareed Khan](#))

Selective accuracy counts value per question asked.

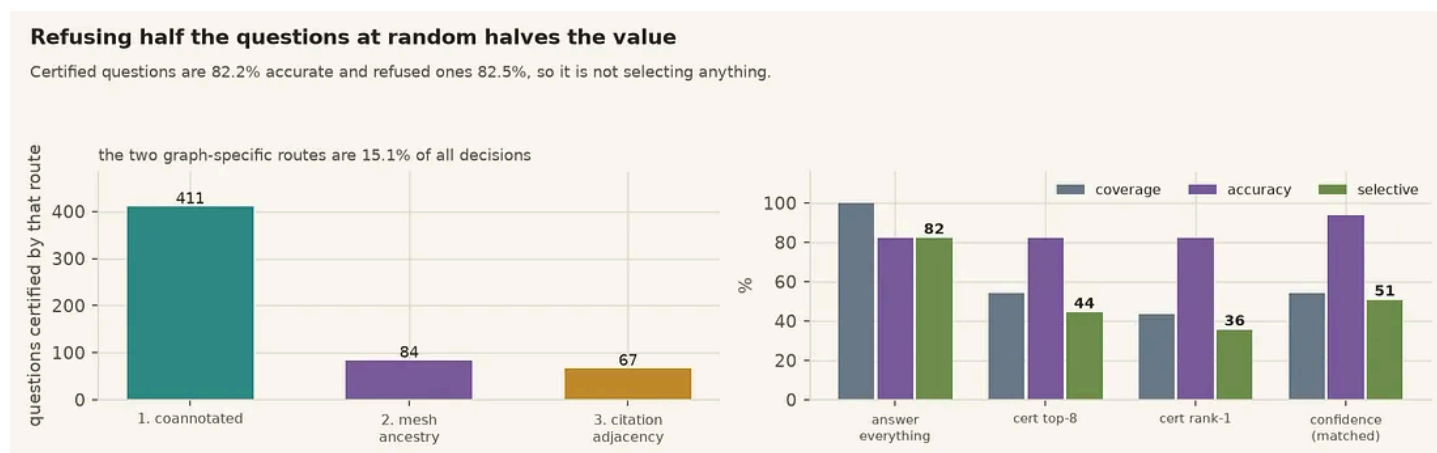


OUTPUT

Selective accuracy (accuracy x coverage) – value per question ASKED:

answer everything	100.0%	82.3%	82.3%
path certification, top-8	54.2%	82.2%	44.5%
path certification, rank-1	43.3%	82.3%	35.7%
confidence, matched coverage	54.2%	93.8%	50.8%

Answering everything scores 82.3% and beats certification outright, because refusing at random halves the value.



Refusing half the questions at random halves the value (Created by [Fareed Khan](#))

This is a refutation of my own central claim, on this benchmark, under this criterion. It tests certification as abstention over a fixed answer. The original design also filtered the evidence set before generating, and that half was not re-run, so it could still improve the answer but not the refusal claim.

What the Graph Does Buy

The graph was not pointless.

- **929,824,202 edges for zero model calls**, against a published figure of roughly 33,000 dollars for a much smaller dataset.
- **Gold retrieved into a sixteen-candidate pool 97.4% of the time**, with 1.3% of residual error attributable to retrieval.
- **An explicit admissibility reason per document**, two hundred and seventy seven by co-annotation, seventy four by citation adjacency, sixty two by MeSH ancestry. A cosine score cannot say why.
- **Every answer traceable to a source path**, with zero fabricated citations reaching the user.

It does not buy the refusal decision.

Reproducing All of This

The expensive part is done. The posterior cache took about fifty seven GPU-minutes, and everything downstream is arithmetic over it.

```
python -m venv .venv && .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python analysis/run_all.py
```

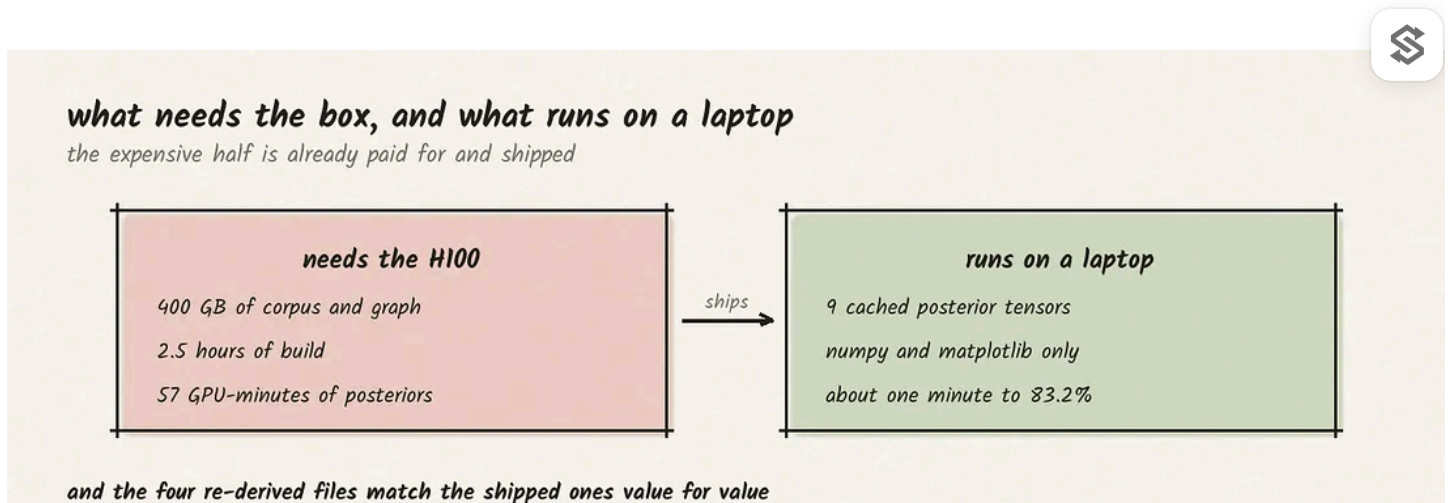
The requirements are numpy and matplotlib.

```
#### OUTPUT ####
=====
ALL ANALYSES COMPLETE
=====
configurations built      139  (137 selectable; oracle + blind excluded)
```

<code>split</code>	DEV 400 / TEST 600
<code>winner</code>	<code>p0 concat ce_k1</code>
<code>no-evidence floor</code>	37.2%
<code>majority-class floor</code>	53.3%
<code>graph certification AUROC</code>	0.500 (chance)
<code>free confidence AUROC</code>	0.810
TEST accuracy (argmax)	82.3%
TEST accuracy	83.2%

reproduces the published 83.2%: YES

The run reproduces the published 83.2%, and **every value matched exactly**. The files are not byte-identical, because JSON key ordering differs, but every number is the same.



What needs the box, and what runs on a laptop (Created by [Fareed Khan](#))

One hazard for the GPU side. The thirty two billion parameter script writes corrected-truncation results to the **old** cache filename, overwriting what the model-scale comparison depends on.

Bugs That Produced Reassuring Wrong Answers

Collected in one place, because they share a signature.

```
#### OUTPUT ####
1. major-topic arrays misaligned
   cause   : two dedup rules, two sort orders, in two separate passes
   symptom : evidence labelled with the wrong article's flags
   fix     : rebuild in one pass + alignment assert + DB spot-check

2. gate 6 scored a concatenation
```

cause : verifier trained on claim-vs-ONE-abstract, fed 8 concatenated
symptom : 0.97 neutrality on true claims -> refused good answers, looked
fix : score against the best single source (0.97 -> 0.02 on an ident

3. ranker cache keyed to first question

cause : cache populated by question 1's pmid set, reused for all
symptom : every later question scored -1.0 and fell back to arbitrary or
fix : one pmid-sorted memmap index + searchsorted

4. citation regex missed multi-PMID

cause : the pattern matched one id per bracket, not a list
symptom : '0 fabricated citations' was vacuous - nothing was being checked
fix : match the bracket, split, rewrite keeping only valid ids

5. head-noun grounding

cause : coverage +8 points looked like a clear win
symptom : precision 0.610 -> 0.535, +103 false positives
fix : measure precision against NLM headings; reject the feature

All five degraded silently. Four of the five made a metric IMPROVE.



All five degraded silently, and four of the five made a metric improve. Add the truncation constant and that is six, all with the same shape.

An end-to-end score cannot say that the thing it improved was the wrong thing, which is the argument for per-layer ground truth.

What Is Still Open

Ranked by measured impact.

1. **Grounding is still the ceiling**, causing 54% of all attrition, and the hybrid linker moved coverage from about 37% to about 60% rather than solving it.
2. **Path existence is not claim truth.** A graph edge encodes aboutness, never polarity, so aspirin and myocardial infarction co-occur whether aspirin helps or harms.
3. **The verifier is scoped to one task and I proved the boundary**, so treat 0.932 as scoped to SciFact.
4. **The benchmark is close to exhausted.** At 82.4% accuracy with gold retrieved 97.4% of the time, it largely measures document identification, which needs no graph.

5. **The cross-encoder reads only the first 900 characters**, the same class of constant as the one that cost fourteen points, and never swept.
6. **Two-hop bridges were never implemented**, and a second hop needs the ranker to be load-bearing first.

Where This Leaves Us

I set out to make hallucination become abstention, and to prove the abstention was enforced rather than hoped for.

The enforcement machinery works and it is cheap. A 929-million-edge graph over forty million documents, built in ten point eight minutes for zero model calls, a grounder that cannot invent an identifier, five gates that run before the generator, and refusal tracking evidence availability causally from 7.5% to 100% across an ablation.

The governance thesis on top of it did not survive, because path certification predicts correctness at chance while the model's own confidence reaches 0.810.

So the architecture I would recommend is graph for retrieval and provenance, calibrated confidence for abstention. I had assigned two jobs to one component.

Every serious error here had the same signature, **a metric improved while the system got worse or stayed the same.**

The failure mode is never exotic, always a plausible number that nobody tried to break.

Thanks for reading this far.



AI

+

Artificial Intelligence

+

Data Science

+

Python

+

AI Agent

+



Подписаться

Автор: Фарид Хан

77 ТЫСЯЧ подписчиков · 0 подписчиков

Я пишу об искусственном интеллекте, <https://www.linkedin.com/in/fareed-khan-dev/>



Подписаться

Опубликовано в Level Up Coding

359 тысяч подписчиков · Опубликовано 1 день назад

Уроки по программированию и новости. Домашняя страница разработчика gitconnected.com && skilled.dev && levelup.dev





